

Politechnika Warszawska

W Y D Z I A Ł E L E K T R Y C Z N Y



Instytut Sterowania i Elektroniki Przemysłowej

Praca dyplomowa magisterska

na kierunku Informatyka Stosowana
w specjalności Inżynieria Oprogramowania

Bot Bowl - nauka sieci neuronowej gry w grę Blood Bowl

inż. Mikołaj Klębowski

numer albumu 299253

promotor

dr inż. Krzysztof Hryniów

WARSZAWA 2026

Bot Bowl - nauka sieci neuronowej gry w grę Blood Bowl

Streszczenie

Praca dotyczy zastosowania metod uczenia ze wzmocnieniem oraz głębokich sieci neuronowych do nauki gry w strategiczną grę planszową Blood Bowl w środowisku symulacyjnym BotBowl. Blood Bowl stanowi szczególnie wymagające środowisko badawcze dla algorytmów sztucznej inteligencji ze względu na bardzo dużą przestrzeń stanów, wysoki współczynnik rozgałęzienia decyzji (branching factor), obecność losowości wynikającej z mechaniki rzutów kośćmi oraz długoterminowe zależności pomiędzy podejmowanymi decyzjami. W takich warunkach tradycyjne algorytmy przeszukiwania drzew decyzyjnych stają się niewystarczające, co sprawia, że uczenie ze wzmocnieniem stanowi obiecujące podejście do konstruowania agentów zdolnych do podejmowania skutecznych strategii.

Celem pracy było porównanie różnych architektur sieci neuronowych wykorzystywanych w agentach uczących się gry w Blood Bowl przy użyciu algorytmów typu actor-critic. W szczególności analizowano wpływ architektury sieci na efektywność procesu uczenia oraz jakość wyuczonej polityki decyzyjnej. W badaniach zastosowano dwa popularne algorytmy uczenia ze wzmocnieniem: Advantage Actor-Critic (A2C) oraz Proximal Policy Optimization (PPO). W obu przypadkach sieć neuronowa pełniła rolę zarówno polityki decyzyjnej (aktor), jak i funkcji wartości stanu (krytyk), przy czym obie części współdzieliły wspólny konwolucyjny moduł ekstrakcji cech.

W pracy szczególną uwagę poświęcono architekturom sieci konwolucyjnych przetwarzających przestrzenną reprezentację stanu gry. Analizowane były różne warianty architektur, w tym modele wykorzystujące klasyczne warstwy konwolucyjne, bloki Inception umożliwiające analizę wieloskalowych zależności przestrzennych, architekturę ResNet wykorzystującą połączenia rezydualne oraz modele łączące oba podejścia w postaci bloków Inception-Residual. Dodatkowo badano wpływ mechanizmu hyper-connections, stanowiącego uogólnienie klasycznych połączeń rezydualnych, który pozwala sieci uczyć się zależności pomiędzy reprezentacjami z różnych głębokości modelu. W architekturach tych zastosowano również techniki stabilizacji uczenia, takie jak Layer Normalization, inicjalizacja wag He, funkcja aktywacji PReLU oraz maskowanie niedozwolonych akcji.

Reprezentacja stanu gry została podzielona na dwa strumienie danych: przestrzenny oraz nieprzestrzenny. Dane przestrzenne reprezentujące układ planszy przetwarzane były przez sieci konwolucyjne, natomiast dane opisujące parametry gry, takie jak licznik tur czy dostępne przerzuty, przetwarzane były przez warstwy w pełni połączone. Następnie obie reprezentacje były łączone w jedną wspólną reprezentację stanu, która stanowiła wejście dla części decyzyjnej sieci.

Eksperymenty przeprowadzono w środowisku BotBowl, które implementuje pełne zasady gry Blood Bowl oraz udostępnia interfejs zgodny ze standardem OpenAI Gym. Proces uczenia polegał na wielokrotnym rozgrywaniu symulowanych meczów pomiędzy agentami, a postęp uczenia oceniano na podstawie kilku metryk, takich jak średnia nagroda (Reward), liczba przyłożeń na epizod (TD/Episode) oraz wskaźnik zwycięstw (Win rate). Najważniejszym kryterium oceny skuteczności agenta był wskaźnik zwycięstw w meczach ewaluacyjnych.

Uzyskane wyniki wskazują, że architektura sieci neuronowej ma istotny wpływ na zdolność agenta do nauki skutecznej strategii gry. W szczególności modele wykorzystujące bardziej zaawansowane mechanizmy ekstrakcji cech przestrzennych wykazują większą stabilność procesu uczenia oraz lepszą zdolność do stopniowej poprawy jakości podejmowanych decyzji. Jednocześnie eksperymenty pokazały, że środowisko Blood Bowl generuje znaczną wariancję wyników ze względu na losowość mechaniki gry oraz rzadkość nagród, co utrudnia jednoznaczną ocenę jakości modeli na podstawie pojedynczych przebiegów treningu.

Wyniki pracy potwierdzają, że projektowanie odpowiedniej architektury sieci neuronowej jest kluczowym elementem skutecznego zastosowania uczenia ze wzmocnieniem w złożonych środowiskach decyzyjnych. Jednocześnie wskazują potencjalne kierunki dalszych badań, obejmujące wykorzystanie głębszych modeli konwolucyjnych, mechanizmów uwagi oraz bardziej zaawansowanych metod uczenia agentów w środowiskach o bardzo dużej przestrzeni decyzji.

Słowa kluczowe: uczenie ze wzmocnieniem, Blood Bowl, BotBowl, sieci neuronowe, PPO, A2C, sieci konwolucyjne.

Bot Bowl - learning the neural network game of Blood Bowl

Abstract

The thesis concerns the application of reinforcement learning methods and deep neural networks to learning how to play the strategic board game Blood Bowl within the BotBowl simulation environment. Blood Bowl constitutes a particularly challenging research environment for artificial intelligence algorithms due to the very large state space, a high decision branching factor, the presence of randomness resulting from dice roll mechanics, and long-term dependencies between decisions. Under such conditions, traditional decision tree search algorithms become insufficient, which makes reinforcement learning a promising approach for constructing agents capable of developing effective strategies.

The objective of this thesis was to compare different neural network architectures used in agents learning to play Blood Bowl with the use of actor-critic type algorithms. In particular, the study analyzed the influence of network architecture on the efficiency of the learning process and the quality of the learned decision policy. Two popular reinforcement learning algorithms were used in the study: Advantage Actor-Critic (A2C) and Proximal Policy Optimization (PPO). In both cases, the neural network served both as the decision policy (actor) and as the state value function (critic), with both components sharing a common convolutional feature extraction module.

Special attention in the thesis was devoted to convolutional network architectures processing the spatial representation of the game state. Various architectural variants were analyzed, including models using classical convolutional layers, Inception blocks enabling the analysis of multi-scale spatial dependencies, the ResNet architecture utilizing residual connections, and models combining both approaches in the form of Inception-Residual blocks. Additionally, the influence of the hyper-connections mechanism was investigated. This mechanism represents a generalization of classical residual connections and allows the network to learn relationships between representations from different depths of the model. The architectures also employed learning stabilization techniques such as Layer Normalization, He weight initialization, the PReLU activation function, and masking of invalid actions.

The representation of the game state was divided into two data streams: spatial and non-spatial. Spatial data representing the board configuration were processed by convolutional networks, whereas data describing game parameters, such as the turn counter or available rerolls, were processed by fully connected layers. The two representations were then combined into a single shared state representation, which served as input for the decision-making part of the network.

The experiments were conducted in the BotBowl environment, which implements the full rules of the Blood Bowl game and provides an interface compatible with the OpenAI Gym standard. The learning process consisted of repeatedly playing simulated matches between agents, and the learning progress was evaluated using several metrics, such as average reward (Reward), number of touchdowns per episode (TD/Episode), and win rate (Win rate). The most important criterion for evaluating the agent's effectiveness was the win rate in evaluation matches.

The obtained results indicate that the neural network architecture has a significant impact on the agent's ability to learn an effective game strategy. In particular, models utilizing more advanced spatial feature extraction mechanisms exhibit greater stability during the learning process and a better ability to gradually improve the quality of decisions. At the same time, the experiments showed that the Blood Bowl environment generates significant variance in results due to the randomness of the game mechanics and the sparsity of rewards, which makes it difficult to unambiguously evaluate model quality based on single training runs.

The results of the thesis confirm that designing an appropriate neural network architecture is a key element for the successful application of reinforcement learning in complex decision-making environments. At the same time, they indicate potential directions for further research, including the use of deeper convolutional models, attention mechanisms, and more advanced methods for training agents in environments with very large decision spaces.

Keywords: reinforcement learning, Blood Bowl, BotBowl, neural networks, PPO, A2C, convolutional networks.

Spis treści

1	Wstęp	9
2	Sieci Neuronowe	11
2.1	Funkcja aktywacji	12
2.2	Architektura sieci neuronowej	14
2.3	Sieci Splotowe	15
2.3.1	Geneza	15
2.3.2	Sposób działania	16
2.4	Techniki uczenia sieci neuronowych	18
2.5	Funkcje kosztu (straty)	20
2.6	Algorytmy optymalizacji	20
2.7	Problemy gradientowe	24
2.8	Techniki stabilizacji i usprawnienia procesu uczenia	26
3	Zestawienie użytych rozwiązań	29
3.1	Algorytmy aktor–krytyk	29
3.2	Advanced Actor Critic	33
3.3	PPO	34
3.4	Współdzielenie części konwolucyjnej	36
3.5	Reprezentacja danych wejściowych	36
3.6	Architektura typu CNN z blokiem Inception	38
3.7	Resnet	39
3.8	Inception-Residual	40
3.9	Hyperconnction	41
3.10	Rozszerzenia architektury sieci i techniki stabilizacji uczenia	44
4	Środowisko testowe	49
4.1	Bloodbowl jako benchmark dla uczenia ze wzmocnieniem.	50
4.2	Środowisko BotBowl	51
5	Wyniki	53
	Wykaz skrótów i symboli	61

Spis rysunków	63
Spis tabel	65

Rozdział 1

Wstęp

W ostatnich latach sztuczna inteligencja, a w szczególności metody oparte na głębokich sieciach neuronowych, stały się jednym z kluczowych obszarów rozwoju współczesnej informatyki. Ich zastosowania obejmują zarówno zadania percepcyjne, takie jak rozpoznawanie obrazów i mowy, jak i problemy decyzyjne występujące w systemach autonomicznych, analizie danych czy optymalizacji procesów przemysłowych. Dynamiczny postęp w tej dziedzinie wynika w dużej mierze z rosnących możliwości obliczeniowych oraz z rozwoju algorytmów umożliwiających skuteczne uczenie modeli na podstawie dużych zbiorów danych. Coraz częściej systemy te są w stanie podejmować złożone decyzje w sposób autonomiczny, bez konieczności bezpośredniego nadzoru człowieka.

Szczególną rolę w tym kontekście odgrywa uczenie ze wzmocnieniem, które stanowi paradygmat uczenia maszynowego skoncentrowany na problemach sekwencyjnego podejmowania decyzji. W przeciwieństwie do uczenia nadzorowanego, gdzie model otrzymuje jednoznaczne pary wejście–wyjście, w uczeniu ze wzmocnieniem agent zdobywa wiedzę poprzez interakcję ze środowiskiem i obserwację konsekwencji własnych działań. Takie podejście pozwala na modelowanie problemów, w których nagroda jest opóźniona w czasie, a decyzje podejmowane na wczesnym etapie wpływają na dalszy przebieg procesu. Z tego względu metody te znajdują zastosowanie m.in. w robotyce, systemach sterowania, logistyce, finansach oraz w grach.

Gry od wielu lat pełnią funkcję istotnych środowisk testowych dla algorytmów sztucznej inteligencji. Ich popularność wynika z połączenia kilku kluczowych cech: jasno zdefiniowanych reguł, możliwości precyzyjnego pomiaru wyników oraz łatwości powtarzania eksperymentów w identycznych warunkach początkowych. Dzięki temu gry umożliwiają obiektywne porównywanie skuteczności różnych metod i architektur modeli uczących się. W historii rozwoju sztucznej inteligencji wielokrotnie wykorzystywano środowiska gier planszowych, karcianych oraz komputerowych jako poligony badawcze, pozwalające na analizę zachowania agentów w warunkach wysokiej złożoności decyzyjnej.

Jednym z najbardziej wymagających aspektów środowisk decyzyjnych jest duża przestrzeń stanów oraz wysoki współczynnik rozgałęzienia możliwych akcji, określany jako branching factor. W takich warunkach liczba potencjalnych trajektorii rozgrywki rośnie wykładniczo wraz z długością sekwencji decyzji, co znacząco utrudnia zarówno eksplorację przestrzeni rozwiązań, jak i stabilne uczenie modeli.

Dodatkowym wyzwaniem bywa obecność losowości, która wprowadza niepewność co do skutków podejmowanych działań i wymusza na agencie uczenie się strategii odpornych na zmienne warunki.

Gra Blood Bowl, będąca turową grą strategiczną opartą na zasadach gry planszowej, stanowi przykład środowiska spełniającego wszystkie powyższe kryteria. Rozgrywka łączy elementy planowania długoterminowego, zarządzania ryzykiem oraz reagowania na zdarzenia losowe, takie jak wyniki rzutów kośćmi. W każdej turze gracz ma do dyspozycji wiele możliwych akcji, których kolejność i wybór mają istotny wpływ na dalszy przebieg meczu. Wysoki poziom złożoności decyzyjnej oraz wieloetapowy charakter gry sprawiają, że Blood Bowl jest wymagającym, a jednocześnie interesującym środowiskiem do badań nad algorytmami uczenia ze wzmocnieniem.

Motywacją do podjęcia tematu niniejszej pracy jest potrzeba lepszego zrozumienia, w jaki sposób różne architektury sieci neuronowych radzą sobie w środowiskach o dużej przestrzeni decyzyjnej i znacznej losowości. W literaturze przedmiotu często prezentowane są wyniki osiągnięte w relatywnie prostych środowiskach, takich jak klasyczne gry Atari czy zadania o ograniczonej liczbie akcji. Tymczasem problemy zbliżone do rzeczywistych zastosowań charakteryzują się znacznie większą złożonością, co może ujawniać istotne różnice pomiędzy poszczególnymi podejściami architektonicznymi i algorytmicznymi.

Celem niniejszej pracy jest porównanie wybranych rozwiązań opartych na uczeniu ze wzmocnieniem w kontekście gry Blood Bowl, ze szczególnym uwzględnieniem wpływu architektury sieci neuronowej na efektywność procesu uczenia. Analizie poddane zostaną różne podejścia do przetwarzania informacji, a także ich zdolność do wypracowywania skutecznych strategii w złożonym środowisku gry. Porównanie to pozwoli na ocenę, w jakim stopniu określone rozwiązania sprzyjają stabilnemu uczeniu oraz szybkiej adaptacji agenta do reguł rozgrywki.

Zakres pracy obejmuje zarówno omówienie teoretycznych podstaw uczenia ze wzmocnieniem i sieci neuronowych, jak i analizę praktycznych aspektów ich zastosowania w środowisku gry. W kolejnych rozdziałach przedstawione zostaną podstawy teoretyczne, środowisko symulacyjne wykorzystywane w badaniach, opis zastosowanych modeli oraz metodyka przeprowadzonych eksperymentów. Całość zostanie zwieńczona analizą uzyskanych wyników oraz wnioskami dotyczącymi przydatności poszczególnych architektur w kontekście złożonych problemów decyzyjnych.

Rozdział 2

Sieci Neuronowe

Idea sztucznych sieci neuronowych narodziła się podczas prób modelowania pracy ludzkiego mózgu. Warren S. McCulloch oraz Walter Pitts w swojej pracy "A logical calculus of the ideas immanent in nervous activity" [14] zaproponowali model sieci neuronowej, jako grafu skierowanego, gdzie węzły to matematyczne modele neuronu takie, że

$$N_i(t+1) = H\left(\sum_{j=1}^n w_{ij}(t)N_j(t) - \Theta_i(t)\right)$$

gdzie:

- N - stan neuronu, może przyjąć wartość 0 lub 1
- H - funkcja skokowa Heaviside'a
- w_{ij} - waga połączenia między wektorem j oraz i .
- Θ - próg wyzwalania, neuron aktywuje się, gdy sygnał wejściowy go przekroczy.

Sieć złożona z takich neuronów jest nazywana dzisiaj perceptronem. Służy ona do zadań klasyfikacji binarnej. Potrafi określić przynależność badanego obiektu do jednej z klas na podstawie wybranych cech. W 1957 roku Frank Rosenblatt opracował pierwszy uczący się perceptron, który stanowił przełom w rozwoju sztucznych sieci neuronowych. Pomimo swojej innowacyjności, model ten miał istotne ograniczenia – nie potrafił rozwiązywać problemów nieliniowych, takich jak klasyczny przykład funkcji XOR, a jego struktura uniemożliwiała zastosowanie metod uczenia opartych na obliczaniu pochodnych. Źródłem tego ograniczenia była funkcja skokowa Heaviside'a, wykorzystywana jako funkcja aktywacji neuronu. Współcześnie zastępuje się ją funkcjami ciągłymi i różniczkowalnymi, takimi jak sigmoid, tangens hiperboliczny czy ReLU (Rectified Linear Unit) wraz z jej modyfikacjami (np. Leaky ReLU, Parametric ReLU), które umożliwiają skuteczne stosowanie algorytmów gradientowych, takich jak propagacja wsteczna błędów. [16] Jak zostało wcześniej wspomniane ten model został stworzony w celu opisanie sposobu działania biologicznego neuronu. Można go uogólnić, pozwalając stworzyć bardziej uniwersalny model matematyczny.

$$y = f \left(\sum_{i=1}^m w_i(t)x_i(t) + b \right)$$

gdzie:

- w_i – waga
- x_i – wejście,
- f – funkcja aktywacji,
- y – wyjście,
- m – liczba wejść,
- b – stała wartość (ang. bias).

Do funkcji aktywacji f przekazywana jest ważona suma sygnałów wejściowych, do której dodaje się opcjonalnie parametr bias. Taki opis umożliwia zastosowanie dowolnej funkcji aktywacji, co pozwala sieci neuronowej modelować nieliniowe zależności.

2.1 Funkcja aktywacji

Jednym z kluczowych elementów każdego neuronu sztucznego jest funkcja aktywacji, która decyduje o tym, w jaki sposób sygnał wejściowy zostanie przekształcony w sygnał wyjściowy. Od jej właściwości zależy zdolność sieci neuronowej do modelowania złożonych, nieliniowych zależności.

Pierwszym powszechnie stosowanym rozwiązaniem była funkcja skokowa Heaviside'a:

$$H(x) = \begin{cases} 0, & x < 0, \\ 1, & x \geq 0. \end{cases}$$

która aktywuje neuron tylko po przekroczeniu określonego progu. Funkcja ta dobrze oddaje ideę neuronu biologicznego, który „uaktywnia się” po przekroczeniu pewnego poziomu pobudzenia, jednak z matematycznego punktu widzenia ma istotną wadę — nie jest różniczkowalna. Jej pochodna jest równa zero dla wszystkich wartości oprócz punktu $x=0$, gdzie jest nieokreślona. W konsekwencji uniemożliwia to stosowanie metod uczenia opartych na gradientach, takich jak propagacja wsteczna błędów.

Aby umożliwić płynne i różniczkowalne przejście między stanami aktywacji, wprowadzono funkcje ciągłe, takie jak sigmoid i tangens hiperboliczny ($\tanh$). Funkcja sigmoidalna ma postać:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

a tangens hiperboliczny:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Obie funkcje zapewniają gładką, różniczkowalną zależność między wejściem a wyjściem neuronu, dzięki czemu możliwe jest stosowanie gradientowych metod optymalizacji. Dodatkowo funkcja $\tanh(x)$ ma zakres $(-1, 1)$, co czyni ją bardziej symetryczną względem zera i ułatwia uczenie w sieciach głębokich. Ich wadą jest jednak zjawisko nasycenia – dla dużych wartości argumentu pochodna staje się bliska zeru, co prowadzi do problemu zanikającego gradientu.

W odpowiedzi na te ograniczenia opracowano funkcję ReLU (Rectified Linear Unit), definiowaną jako:

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} 0, & x \leq 0, \\ x, & x > 0. \end{cases}$$

Funkcja ta jest ciągła w całej swojej dziedzinie, również w punkcie $x=0$, lecz nie jest różniczkowalna w tym punkcie — lewa pochodna wynosi 0, a prawa 1. W praktyce jednak przyjmuje się, że funkcja jest „różniczkowalna prawie wszędzie”, co pozwala skutecznie stosować ją w uczeniu sieci metodami gradientowymi. ReLU eliminuje zjawisko zanikającego gradientu dla dodatnich wartości wejść, dzięki czemu znacząco przyspiesza uczenie głębokich modeli. Współczesne warianty, takie jak Leaky ReLU, Parametric ReLU czy ELU (Exponential Linear Unit), pozwalają dodatkowo ograniczyć problem tzw. martwych neuronów, które przestają reagować na sygnał uczący.

W sztucznej sieci neuronowej neurony są zorganizowane w warstwy, z których każda przetwarza sygnały z poprzedniej i przekazuje wynik dalej. Typowa struktura to warstwa wejściowa, jedna lub więcej warstw ukrytych oraz warstwa wyjściowa. Zazwyczaj warstwy są ze sobą w pełni połączone (ang. *fully connected*), co oznacza, że każdy neuron danej warstwy jest połączony z każdym neuronem warstwy następnej.

Z punktu widzenia matematycznego, każda warstwa sieci realizuje pewną funkcję przekształcającą wektor wejściowy w nową reprezentację:

$$h^{(l)} = f^{(l)} \left(W^{(l)} h^{(l-1)} + b^{(l)} \right), \quad (1)$$

gdzie:

- $h^{(l)}$ – wektor wyjść (aktywacji) z warstwy l ,
- $W^{(l)}$ – macierz wag warstwy l ,
- $b^{(l)}$ – wektor przesunięć (*bias*),
- $f^{(l)}(\cdot)$ – funkcja aktywacji danej warstwy.

Cała sieć neuronowa może być opisana jako złożenie (kompozycja) takich funkcji:

$$f(x) = f^{(L)} \left(f^{(L-1)} \left(\dots f^{(1)}(x) \dots \right) \right), \quad (2)$$

gdzie L oznacza liczbę warstw w sieci, a x jest wektorem wejściowym.

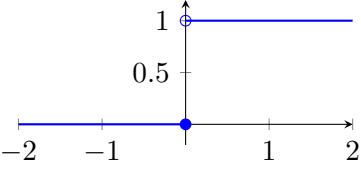
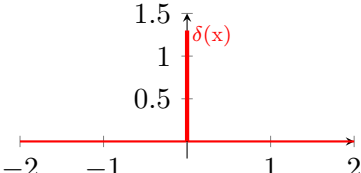
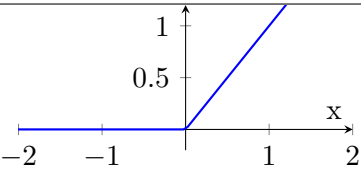
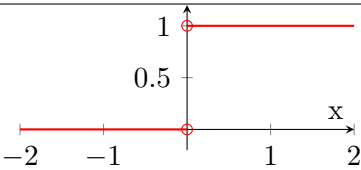
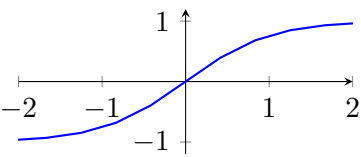
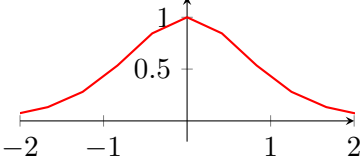
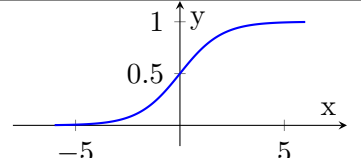
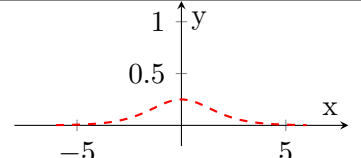
Porównanie funkcji i ich pochodnych	
Funkcja Heaviside $H(x)$	Pochodna $H(x)$
	
Funkcja ReLU(x)	Pochodna ReLU(x)
	
Tangens hiperboliczny $\tanh(x)$	Pochodna $\tanh(x)$
	
$\sigma(x)$	Pochodna $\sigma(x)$
	

Tabela 1. Porównanie funkcji aktywacji oraz ich pochodnych

2.2 Architektura sieci neuronowej

Głębokość i zdolność reprezentacyjna

Określenie *płytką sieć* (*shallow network*) odnosi się do modeli z jedną warstwą ukrytą, natomiast *głębokie sieci* (*deep networks*) zawierają wiele takich warstw. Głębsza architektura pozwala na hierarchiczne przekształcanie danych wejściowych — od prostych cech niskiego poziomu po coraz bardziej abstrakcyjne reprezentacje. Dzięki temu kolejne warstwy mogą uczyć się złożonych zależności, które trudno byłoby odwzorować w jednej transformacji.

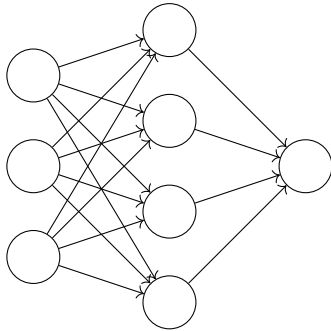
Zależność pomiędzy głębokością sieci a jej zdolnością reprezentacyjną opisuje tzw. **twierdzenie o uniwersalnej aproksymacji**. Mówi ono, że sieć z jedną warstwą ukrytą i odpowiednią liczbą neuronów jest w stanie przybliżyć dowolną funkcję ciągłą na zwartym zbiorze [3, 9]. W praktyce jednak takie sieci wymagają ogromnej liczby neuronów i parametrów, co czyni je nieefektywnymi obliczeniowo. Sieci głębokie uzyskują podobną moc aproksymacyjną przy znacznie mniejszej liczbie parametrów, rozkładając złożoność obliczeń na wiele etapów przetwarzania.

Formalnie działanie sieci z jedną warstwą ukrytą można zapisać jako:

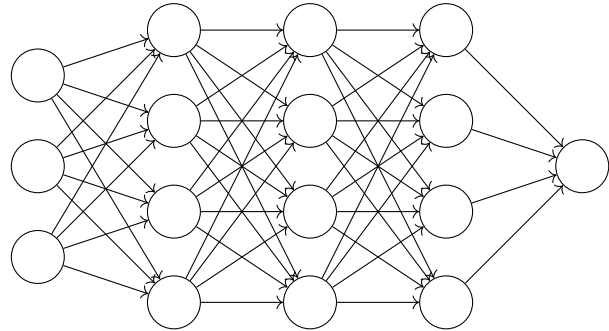
$$f(x) = \sum_{i=1}^N \alpha_i \sigma(w_i^T x + b_i), \quad (3)$$

gdzie $\sigma(\cdot)$ jest nieliniową funkcją aktywacji, a α_i , w_i , b_i stanowią parametry uczone w procesie treningu.

Płytką sieć neuronową



Głęboka sieć neuronowa



Rysunek 1. Porównanie płytkiej i głębokiej sieci neuronowej z wyrównanym położeniem neuronów wyjściowych.

Badania [15, 20, 26] wykazały istnienie tzw. **hierarchii głębokości**, zgodnie z którą zwiększanie liczby warstw jest znacznie bardziej efektywne niż ich poszerzanie. Dla pewnych klas funkcji ograniczenie głębokości prowadzi do wykładniczego wzrostu liczby neuronów wymaganych do osiągnięcia tej samej jakości aproksymacji. Zależność tę można opisać asymptotycznie:

$$N_{\text{shallow}} = \mathcal{O}(2^k), \quad N_{\text{deep}} = \mathcal{O}(k^3), \quad (4)$$

gdzie k oznacza głębokość sieci. Zwiększenie głębokości modelu pozwala zatem uzyskać porównywalną zdolność aproksymacji określonych klas funkcji przy istotnie mniejszej liczbie neuronów i parametrów.

2.3 Sieci Splotowe

2.3.1 Geneza

Sieci splotowe (ang. Convolutional Neural Networks, CNN) stanowią jeden z najważniejszych rodzajów sieci głębokiego uczenia. Ich powstanie inspirowane było budową kory wzrokowej, w której poszczególne neurony reagują na proste wzorce w ograniczonych fragmentach pola widzenia. W odróżnieniu od klasycznych wielowarstwowych perceptronów, sieci splotowe wykorzystują lokalne filtry do automatycznego wydobywania cech z danych wejściowych.

Każdy filtr przetwarza jedynie lokalny fragment wejścia, co umożliwia wykrywanie wzorców przestrzennych, takich jak krawędzie, linie, tekstury oraz bardziej złożone struktury. Mechanizm ten

pozwała na efektywne modelowanie zależności lokalnych, charakterystycznych dla danych o strukturze przestrzennej.

Dzięki współdzieleniu wag, te same filtry są stosowane w różnych położeniach wejścia, co zapewnia możliwość wykrywania tych samych wzorców w różnych położeniach danych wejściowych oraz znacząco redukuje liczbę parametrów modelu. W rezultacie sieci splotowe wykazują dobrą zdolność generalizacji.

W kontekście gier o dużej złożoności, takich jak BloodBowl, sieci splotowe pełnią rolę ekstraktora cech przestrzennych, przekształcając reprezentację planszy gry w wielopoziomowy zbiór cech o rosnącym stopniu abstrakcji umożliwiającą agentowi lepsze zrozumienie sytuacji na boisku. Tego typu architektury są powszechnie stosowane w algorytmach uczenia ze wzmocnieniem, takich jak A2C czy PPO, gdzie służą do przetwarzania obrazu stanu gry przed podjęciem decyzji przez agenta. [13]

2.3.2 Sposób działania

Podstawowym elementem sieci splotowej jest warstwa konwolucyjna, oparta na operacji splotu. Operacja ta polega na obliczaniu lokalnych sum ważonych pomiędzy fragmentami danych wejściowych a wagami filtra, co prowadzi do powstania mapy aktywacji uwypuklającej obecność określonych wzorców.

$$y(i) = (x * w)(i) = \sum_{m \in \mathcal{M}} x(i + m) w(m),$$

- i - wektor indeksów wyjścia
- m - wektor przesunięć w przestrzeni jądra
- $x(i + m)$ - fragment danych wejściowych
- $w(m)$ - wagi jądra
- $y(i)$ - wartości wyjściowe w punkcie i

W przypadku danych dwuwymiarowych, takich jak plansza gry, stosuje się najczęściej filtry dwuwymiarowe, dla których operacja splotu przyjmuje postać.

$$y(i, j) = (x * w)(i, j) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x(i + m, j + n) w(m, n),$$

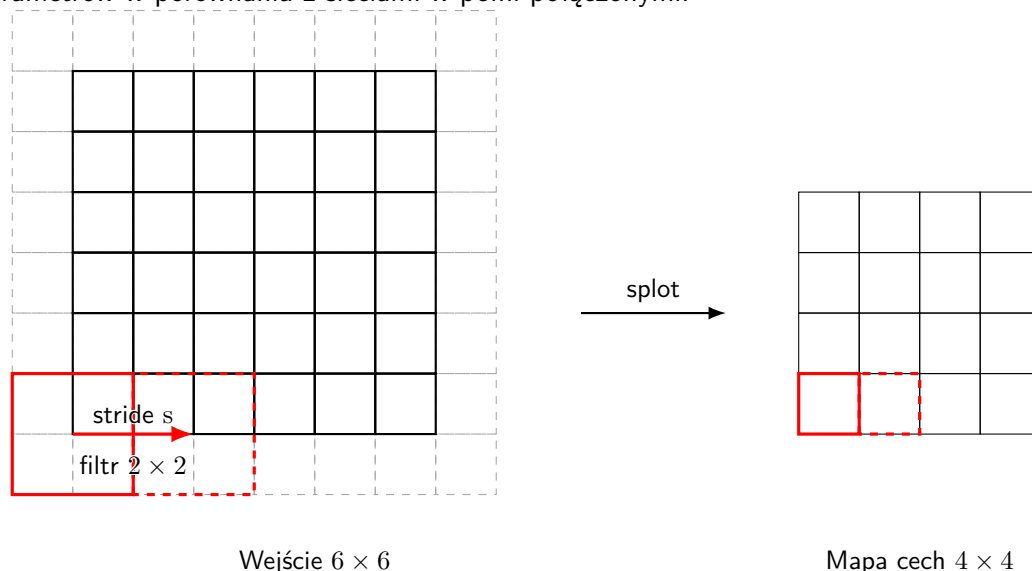
Filtr jest przesuwany po danych wejściowych z określonym krokiem (*stride*), a wynikiem tej operacji jest mapa cech (*feature map*), wskazująca lokalizację występowania danego wzorca.

Struktura warstwy konwolucyjnej

Każda warstwa konwolucyjna składa się z:

- zbioru filtrów (jąder) – niewielkich macierzy uczących się rozpoznawania określonych cech obrazu lub stanu gry,

- kroku przesuwania filtra (stride) – definiuje, co ile pól filtr jest przesuwany
 - mechanizmu uzupełniania brzegów (*padding*) – pozwalającego kontrolować rozmiar map cech.,
- Kluczową własnością sieci splotowych jest współdzielenie wag, dzięki czemu każdy filtr jest stosowany identycznie w całej przestrzeni wejścia. Pozwala to na znaczące ograniczenie liczby parametrów w porównaniu z sieciami w pełni połączonymi.



Mapy cech i hierarchia reprezentacji

Każdy filtr w warstwie konwolucyjnej uczy się wykrywać określony wzorec np pola przez które przechodząc zawodnik może zostać zatrzymany.

Kolejne warstwy CNN tworzą hierarchię reprezentacji:

- pierwsze warstwy rozpoznają proste cechy lokalne
- środkowe warstwy łączą proste cechy w bardziej złożone struktury
- ostatnie warstwy wydobywają informacje semantyczne - np. pola o niskim ryzyku negatywnych efektów

Wynik działania ostatnich warstw konwolucyjnych jest często spłaszczany (*flattening*) i przekazywany do klasycznych warstw gęstych, które dokonują końcowej klasyfikacji lub decyzji.

Warstwy wspierające

W celu zwiększenia efektywności uczenia i zdolność generalizacji, w sieciach konwolucyjnych stosuje się dodatkowe mechanizmy.

- Pooling - redukuje rozdzielczość map cech poprzez agregację lokalnych fragmentów.
- Batch Normalization – normalizuje wartości aktywacji w obrębie mini-batcha, stabilizując proces uczenia i przyspieszając zbieżność.
- Dropout – losowo „wyłącza” część neuronów w czasie treningu, co zapobiega przeuczeniu (*overfitting*)

Połączenie operacji splotu, nieliniowych funkcji aktywacji oraz warstw wspomagających umożliwia budowę głębokich hierarchii reprezentacji przy zachowaniu efektywności obliczeniowej.

2.4 Techniki uczenia sieci neuronowych

Uczenie sieci neuronowych to proces, podczas którego model dopasowuje swoje parametry w taki sposób, aby minimalizować błąd predykcji. Efektywność tego procesu decyduje o zdolności modelu do generalizacji, czyli poprawnego działania na podstawie danych, których nie widział podczas treningu.

Uczenie odbywa się w sposób iteracyjny poprzez modyfikację wag modelu w kierunku przeciwnym do gradientu funkcji kosztu. Każdy cykl uczenia (epoka) obejmuje cztery główne etapy:

- Propagacja w przód (*forward pass*) – dane wejściowe przechodzą przez kolejne warstwy modelu, aż do uzyskania przewidywania $\hat{y}$.
- Obliczenie błędu (*loss*) – funkcja kosztu $(\cdot, \hat{y})$ mierzy różnicę między wartością przewidywaną a oczekiwaną.
- Propagacja wsteczna (*backpropagation*) – na podstawie reguły łańcuchowej obliczane są gradienty funkcji kosztu względem wag.
- Aktualizacja wag (*update*) – parametry są modyfikowane zgodnie z wybranym algorytmem optymalizacji, np. SGD lub Adamem.

Na podstawie sposobu pozyskiwania informacji zwrotnej i dostępności danych uczących wyróżnia się trzy główne paradygmaty uczenia: uczenie nadzorowane, uczenie nienadzorowane oraz uczenie ze wzmocnieniem. [2]

Uczenie nadzorowane

Uczenie nadzorowane (*supervised learning*) jest najczęściej stosowanym paradygmatem uczenia sieci neuronowych. Zakłada ono, że dostępny jest zbiór danych uczących składający się z par (x, y) , gdzie x reprezentuje wektor cech opisujących przykład, a y jest przypisaną do niego etykietą lub wartością wyjściową. Celem modelu jest nauczenie się odwzorowania $f: x \rightarrow y$ w taki sposób, aby przewidywane wyjście $\hat{y}$ było jak najbardziej zbliżone do wartości rzeczywistej y [5].

W ramach uczenia nadzorowanego wyróżnia się dwa główne typy zadań:

- **Klasyfikacja** – gdy etykiety y należą do skończonego zbioru klas, a model ma za zadanie przypisać dany przykład do jednej z nich. Przykłady zastosowań obejmują rozpoznawanie obrazów, klasyfikację dokumentów tekstowych czy diagnozowanie chorób na podstawie danych medycznych.
- **Regresja** – gdy etykieta y jest wartością liczbową, a celem jest przewidzenie wartości ciągłej. Do typowych przykładów należą prognozowanie cen, przewidywanie popytu czy estymacja parametrów fizycznych w eksperymentach naukowych.

Dzięki swojej uniwersalności, uczenie nadzorowane jest stosowane w szerokim zakresie dziedzin, od rozpoznawania mowy i analizy obrazu, przez systemy rekomendacyjne, aż po prognozy finansowe i modelowanie procesów przemysłowych.

Uczenie nienadzorowane

Uczenie nienadzorowane (*unsupervised learning*) obejmuje metody, w których model uczy się wyłącznie na podstawie danych wejściowych x , bez dostępu do odpowiadających im etykiet wynikowych. Celem jest odkrycie wewnętrznej struktury danych, zależności pomiędzy przykładami lub reprezentacji pozwalających na ich bardziej efektywne opisanie [5].

W ramach uczenia nienadzorowanego wyróżnić można kilka głównych typów zadań:

- **Grupowanie (*clustering*)** – podział zbioru danych na zbiory w taki sposób, aby przykłady w tym samym zbiorze były do siebie możliwie podobne, a między nimi różnice jak najbardziej wyraźne. Przykładami takich algorytmów są *k-means*, DBSCAN czy sieci Kohonena (SOM) [12].
- **Redukcja wymiarów** – przekształcenie danych do przestrzeni o mniejszej liczbie wymiarów przy zachowaniu jak największej ilości informacji. Popularne metody to analiza głównych składowych (PCA) czy autoenkodery.
- **Uczenie reprezentacji** – poszukiwanie wewnętrznych opisów danych (tzw. reprezentacji ukrytych), które mogą ułatwić dalsze zadania, np. klasyfikację lub generowanie nowych przykładów. Przykładem są modele generatywne, takie jak *Variational Autoencoder* (VAE) czy *Generative Adversarial Networks* (GAN) [6].

Uczenie nienadzorowane znajduje zastosowanie m.in. w segmentacji obrazów, eksploracji danych, kompresji informacji, wyszukiwaniu podobnych obiektów, detekcji anomalii czy jako etap wstępny w uczeniu nadzorowanym (tzw. *pretraining*).

Uczenie ze wzmocnieniem

Uczenie ze wzmocnieniem (*reinforcement learning*, RL) to proces, w którym agent uczy się poprzez interakcję ze środowiskiem. W każdym kroku czasowym t agent obserwuje stan s_t , wybiera akcję a_t , otrzymuje nagrodę r_t i przechodzi do kolejnego stanu s_{t+1} . Celem jest maksymalizacja zdyskontowanej sumy nagród:

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$$

gdzie $\gamma \in (0, 1)$ to współczynnik dyskontowania, który określa znaczenie przyszłych nagród. Uczenie ze wzmocnieniem znajduje zastosowanie w problemach decyzyjnych – w tym w robotyce, sterowaniu i grach strategicznych, takich jak BloodBowl, Go czy StarCraft, gdzie sieci neuronowe pełnią rolę polityki decyzyjnej (*policy network*).

2.5 Funkcje kosztu (straty)

Funkcja kosztu (ang. *loss function*, *cost function*) pełni kluczową rolę w procesie uczenia sieci neuronowych, ponieważ stanowi ilościową miarę różnicy pomiędzy przewidywaniami modelu $\hat{y}$ a wartościami oczekiwanymi y . Jej wartość stanowi miarę jakości działania modelu dla danych uczących

Na podstawie funkcji kosztu algorytm optymalizacji wyznacza gradienty, które informują, w jaki sposób należy modyfikować wagi sieci, aby stopniowo zmniejszać popełniany błąd. Proces ten prowadzi do iteracyjnego dostosowywania parametrów modelu w kierunku minimalizacji wartości funkcji straty [5]. Dobór odpowiedniej funkcji kosztu zależy od rodzaju rozwiązywanego problemu oraz charakteru danych wyjściowych.

W praktyce można wyróżnić dwie główne grupy funkcji kosztu:

- **Funkcje kosztu dla klasyfikacji** – stosowane, gdy zadaniem modelu jest przypisanie przykładu do jednej z klas:
 - *Entropia krzyżowa* (*cross-entropy loss*) – powszechnie używana w zadaniach klasyfikacji wieloklasowej i binarnej. Funkcja ta silnie karze błędne przewidywania o wysokim poziomie pewności.
 - *Hinge loss* – często stosowana w klasyfikatorach typu SVM, promuje dużą marginesową separację pomiędzy klasami.
- **Funkcje kosztu dla regresji** – używane, gdy przewidywane są wartości ciągłe:
 - *Mean Squared Error (MSE)* – średnia wartość kwadratu różnic pomiędzy przewidywaniami a wartościami rzeczywistymi; silnie penalizuje duże błędy.
 - *Mean Absolute Error (MAE)* – średnia wartość bezwzględna błędów; mniej wrażliwa na wartości odstające niż MSE.

Dobór funkcji kosztu powinien uwzględniać:

- **Charakter problemu** – np. klasyfikacja binarna vs. wieloklasowa, regresja liniowa vs. nieliniowa.
- **Wrażliwość na wartości odstające** – MSE jest silnie wrażliwe na anomalie, MAE bardziej odporne.
- **Interpretowalność** – niektóre funkcje, takie jak log-loss, mają bezpośrednie powiązanie z teorią prawdopodobieństwa.
- **Stabilność numeryczna** – przy dużej liczbie klas lub ekstremalnych wartościach wyjściowych warto stosować wersje funkcji kosztu zoptymalizowane pod kątem obliczeń numerycznych.

Ostateczny wybór funkcji straty ma istotny wpływ na przebieg procesu uczenia, tempo konwergencji oraz końcową jakość modelu.

2.6 Algorytmy optymalizacji

Kluczowym elementem procesu uczenia są Algorytmy optymalizacji. Odpowiadają one za aktualizację parametrów modelu w celu minimalizacji wartości funkcji straty. W praktyce optymalizacja odbywa

się iteracyjnie, a parametry modelu są aktualizowane zgodnie z kierunkiem przeciwnym do gradientu funkcji kosztu względem wag. [5]

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t)$$

gdzie η oznacza współczynnik uczenia (learning rate). Zbyt duża wartość η może powodować niestabilność uczenia, a zbyt mała – spowolnienie konwergencji. Dlatego opracowano szereg metod adaptacyjnych i usprawnień klasycznego spadku gradientu.

Metoda najszybszego spadku

Podstawowy algorytm optymalizacji, stosowany zarówno w wersji pełnej (batch gradient descent), jak i stochastycznej (stochastic gradient descent, SGD). W przypadku danych dzielonych na partie (mini-batches), aktualizacja parametrów przyjmuje postać:

$$\theta_{t+1} = \theta_t - \eta \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L_i(\theta_t)$$

gdzie m to liczba próbek w partii danych.

W praktyce metoda spadku gradientu występuje w trzech głównych wariantach, różniących się sposobem obliczania gradientu. W pełnym spadku gradientu (batch gradient descent) gradient wyznacza się na podstawie całego zbioru treningowego, co daje dokładny kierunek spadku, ale jest obliczeniowo kosztowne i mało praktyczne przy dużych danych. W przeciwieństwie do tego, stochastyczny spadek gradientu (stochastic gradient descent, SGD) aktualizuje wagi po każdej próbce danych, co znacząco przyspiesza uczenie i umożliwia ucieczkę z minimów lokalnych, jednak wprowadza losowe oscylacje w kierunku gradientu. Kompromisem pomiędzy tymi podejściami jest mini-batch gradient descent, który wykorzystuje niewielkie partie danych (zazwyczaj od kilkunastu do kilkuset przykładów). Takie podejście poprawia stabilność uczenia i umożliwia efektywne wykorzystanie równoległości obliczeń, np. na procesorach GPU.

Choć klasyczna metoda spadku gradientu jest intuicyjna i prosta w implementacji, ma istotne ograniczenia. Jest bardzo wrażliwa na dobór współczynnika uczenia i może łatwo utknąć w minimach lokalnych lub punktach siodłowych. Ponadto w obszarach o silnie nachylonych dolinach funkcji kosztu często występują oscylacje, które spowalniają konwergencję

Spadek gradientu z momentem (Momentum)

Metoda *momentum* to usprawnienie klasycznego spadku gradientu, które wprowadza do aktualizacji wag składnik bezwładności (momentum) odpowiednio skalowany przez parametr γ . Pozwala to na użycie większego współczynnika uczenia. Tempo konwergencji znacząco wzrasta, w szczególności w trudnych obszarach funkcji kosztu z dużą ilością minimów lokalnych.

$$\begin{aligned}1 &= \gamma + \eta \\ \mathbf{v}_t &= \gamma \mathbf{v}_{t-1} + \eta \nabla_{\theta} L(\theta_t), \\ \theta_{t+1} &= \theta_t - \mathbf{v}_t.\end{aligned}$$

- η — learning rate (wielkość kroku).
- $\gamma \in [0, 1)$ — momentum coefficient, określa stopień zachowania poprzednich gradientów.

Efektom jest filtr wygładzający historię gradientów, co przyspiesza konwergencję i redukuje oscylacje, a także ułatwia przeciskanie przez lokalne minima.

Metoda przyspieszonego gradientu

Metoda przyspieszonego gradientu, nazywana metodą Nestorova od nazwiska autora, jest dalszym usprawnieniem metody z momentem, w którym gradient jest obliczany nie w bieżącym położeniu parametrów, a w przewidywanym „przyszłym” położeniu modelu. Dzięki temu aktualizacja jest bardziej precyzyjna i stabilna, co pozwala uniknąć nadmiernego przeskoczenia minimum [18]

$$\begin{aligned}\mathbf{v}_t &= \gamma \mathbf{v}_{t-1} + \eta \nabla_{\theta} L(\theta_t - \gamma \mathbf{v}_{t-1}), \\ \theta_{t+1} &= \theta_t - \mathbf{v}_t.\end{aligned}$$

W praktyce metoda Nesterova jest bardziej stabilna od klasycznego momentum i często prowadzi do szybszej konwergencji, zwłaszcza w sieciach głębokich

RMSProp (Root Mean Square Propagation)

RMSProp to metoda optymalizacji z adaptacyjnymi współczynnikami uczenia. Jest ona modyfikacją optymalizatora AdaGrad (Adaptive Gradient). Algorytmy tego typu dostosowują tempo uczenia dla każdego parametru indywidualnie. [8].

Aktualizacje w RMSProp zapisuje się jako:

$$\begin{aligned}V_t &= \beta V_{t-1} + (1 - \beta) (\nabla_{\theta} L(\theta_t))^2, \\ \theta_{t+1} &= \theta_t - \frac{\eta}{\sqrt{V_t + \epsilon}} \nabla_{\theta} L(\theta_t),\end{aligned}$$

gdzie:

- β (typowo ok. 0.9) — współczynnik wygładzania,
- ϵ — mała stała dla stabilności numerycznej i uniknięcia dzielenia przez zero
- η — współczynnik uczenia.

Dzięki temu zwiększamy tempo nauki parametrów, których gradient jest mały oraz zwalnimy proces nauki parametrów z porównywalnie stromym gradientem. Chroni to sieć przed pominięciem optimum w regionie o stromym gradiencie oraz przyspiesza proces nauki w przeciwnym przypadku.

Zalety RMSProp:

- Utrzymuje adaptacyjny krok uczenia bez jego nadmiernej redukcji,
- Poprawia zbieżność w problemach o zmiennych gradientach — często stosowany w RNN.

Wady:

- Wymaga starannego doboru hiperparametrów,
- Może być mniej skuteczny w przypadku ekstremalnie rzadkich gradientów — wtedy lepszy jest Adam.

Adam (Adaptive Moment Estimation)

Adam stara się łączyć zalety optymalizatorów momentum i RMSProp. Uwzględnia kierunek zmian gradientu i wygładza aktualizacje tak jak momentum oraz adaptacyjnie skaluje krok uczenia względem wariancji gradientów jak RMSProp. Z tego powodu jest on obecnie, wraz z wariantami, najpopularniejszym optymalizatorem sieci neuronowych.

W Adamie dla każdego parametru θ śledzi się dwa momenty:

Pierwszy moment, średnia gradientów

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

Drugi moment, średnia kwadratów gradientów

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

Ze względu na początkowe wartości zerowe m_t oraz v_t wprowadza się poprawki biasu. Dzięki nim na początku uczenie nie jest spowolnione.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2,$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Finalna aktualizacja wag ma postać

$$\theta_{t+1} = \theta_t - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

gdzie $g_t = \nabla_{\theta} L(\theta_t)$, β_1 , β_2 są współczynnikami wygładzania (np. 0.9 i 0.999), a ϵ to mała wartość stabilizująca.

Zalety:

- Adaptacyjny krok dla każdego parametru,
- Efektywny obliczeniowo i pamięciowo,
- Działa dobrze przy danych z szumem, gradientach niestabilnych.

Wady:

- Możliwe problemy ze zbieżnością w funkcjach wypukłych,

- Czasem gorsza generalizacja niż klasyczne SGD z momentum,
- Wymaga dostrojenia wielu hiperparametrów.

[11]

2.7 Problemy gradientowe

Wraz ze wzrostem głębokości sieci neuronowych proces uczenia staje się coraz bardziej podatny na problemy związane z propagacją błędu i stabilnością optymalizacji. Kluczowym czynnikiem wpływającym na skuteczność uczenia jest sposób, w jaki gradienty funkcji kosztu rozchodzą się przez kolejne warstwy modelu. Jeżeli wartości te zanikają lub eksplodują, uczenie głębokiej sieci staje się nieefektywne lub wręcz niemożliwe. Poniżej przedstawiono najważniejsze zjawiska i techniki ich łagodzenia

Nasycenie

Zjawisko nasycenia występuje, gdy dla dużych wartości wejścia funkcja aktywacji osiąga swoje asymptotyczne maksimum lub minimum, a jej pochodna staje się bliska zeru. W takich warstwach gradient błędu praktycznie nie przepływa dalej, co prowadzi do spowolnienia lub całkowitego zatrzymania procesu uczenia.

Typowym przykładem są funkcje sigmoidalne i tangens hiperboliczny, które dla dużych wartości $|x|$ przestają reagować na zmiany sygnału wejściowego. Wprowadzenie funkcji ReLU i jej wariantów rozwiązało ten problem dla dodatnich wartości argumentu, choć nadal może prowadzić do tzw. martwych neuronów – przypadków, w których gradient po stronie ujemnej jest stale równy zero.

Dodatkowe techniki, takie jak Leaky ReLU czy ELU, wprowadzają niewielkie nachylenie w obszarze ujemnym, co pozwala zachować przepływ gradientu i utrzymać aktywność neuronów w całej sieci.

Skośność rozkładu aktywacji

Kolejnym problemem, często pomijanym, jest skośność rozkładu aktywacji (ang. bias shift). Występuje on, gdy funkcje aktywacji generują wyłącznie dodatnie wartości, jak w przypadku sigmoidy, której zakres wynosi $(0, 1)$. W takiej sytuacji średnia wartość aktywacji jest dodatnia, co powoduje przesunięcie rozkładu wejść kolejnych warstw w kierunku dodatnich wartości. W efekcie sieć pracuje w obszarze nasycenia funkcji aktywacji, a gradienty stopniowo zanikają.

Skośność rozkładu aktywacji utrudnia także utrzymanie stabilnego przepływu sygnału i powoduje, że wagi w kolejnych warstwach są zmuszane do kompensowania przesunięć statystycznych. Rozwiązaniem tego problemu jest stosowanie funkcji aktywacji o zakresie symetrycznym względem zera, takich jak $\tanh(x)$, lub metod normalizacji danych, które utrzymują średnią aktywacji blisko zera (np. Batch Normalization).

Zanikanie gradientu

Zjawisko zanikającego gradientu polega na tym, że wartości gradientów w trakcie propagacji wstecznej stają się coraz mniejsze w miarę przechodzenia przez kolejne warstwy. W konsekwencji wcześniejsze warstwy sieci uczą się bardzo wolno, podczas gdy końcowe – położone bliżej wyjścia – aktualizują swoje wagi znacznie szybciej. Problem ten wynika głównie z własności funkcji aktywacji, takich jak sigmoida lub tangens hiperboliczny, których pochodne przyjmują wartości bliskie zeru dla dużych wartości argumentu. Dla przykładu, w przypadku tangensa hiperbolicznego:

$$\tanh'(x) = 1 - \tanh^2(x)$$

co oznacza, że dla $|x| > 3$ pochodna staje się praktycznie zerowa. Jeżeli gradient pochodnej jest mniejszy niż jeden, a sieć zawiera wiele warstw, iloczyn kolejnych pochodnych szybko dąży do zera, przez co sygnał błędu praktycznie nie dociera do wcześniejszych warstw.

Aby przeciwdziałać zanikaniu gradientu, wprowadza się funkcje aktywacji, które nie nasycają się dla dodatnich wartości argumentu, takie jak ReLU i jej warianty (LeakyReLU, PReLU, ELU). Dodatkowo stosuje się odpowiednie metody inicjalizacji wag (np. He lub Xavier) oraz techniki normalizacji, które utrzymują rozkład aktywacji w kontrolowanym zakresie. [4]

Eksplodujący gradient

Przeciwnym zjawiskiem jest eksplodujący gradient, który występuje wtedy, gdy wartości gradientów w trakcie propagacji wstecznej stają się coraz większe, często osiągając wartości prowadzące do niestabilności numerycznej. W takim przypadku wagi sieci zaczynają przyjmować bardzo duże wartości, co skutkuje rozbieżnością funkcji kosztu i przerwaniem procesu uczenia. Problem ten występuje szczególnie często w głębokich sieciach rekurencyjnych oraz w modelach o źle dobranych współczynnikach uczenia.

Jednym z najczęściej stosowanych sposobów zapobiegania eksplozji gradientów jest ograniczanie ich normy (ang. gradient clipping), które polega na skalowaniu wektora gradientu, jeśli jego długość przekracza ustalony próg τ :

$$g \leftarrow \frac{g}{\max\left(1, \frac{\|g\|}{\tau}\right)}$$

gdzie:

- g — wektor gradientu
- $\|g\|$ - jego norma (najczęściej euklidesowa, czyli 2)
- τ - ustalony próg (np. 1.0)

Dzięki temu uczenie pozostaje stabilne nawet w sytuacjach, gdy lokalny gradient przyjmuje bardzo duże wartości. Innym sposobem łagodzenia tego problemu jest obniżenie współczynnika uczenia lub

zastosowanie metod optymalizacji adaptacyjnej, takich jak RMSProp czy Adam, które automatycznie dopasowują krok optymalizacji dla każdego parametru.

2.8 Techniki stabilizacji i usprawnienia procesu uczenia

Występowanie problemów takich jak zanikający gradient, eksplodujący gradient czy nasycenie funkcji aktywacji powoduje, że odpowiednie metody stabilizacji są niezbędne do osiągnięcia zbieżności. Współczesne sieci wykorzystują szereg technik, które poprawiają przepływ gradientu, ograniczają przeuczenie oraz przyspieszają proces optymalizacji. Do najczęściej stosowanych należą: Batch Normalization, Dropout oraz Learning Rate Scheduling.

Batch Normalization

Jedną z najważniejszych metod poprawiających stabilność uczenia jest Batch Normalization, zaproponowana przez Ioffe i Szegedy'ego w 2015 roku. Jej głównym celem jest utrzymanie stabilnego rozkładu aktywacji w kolejnych warstwach poprzez normalizację danych wejściowych do każdej z nich. W praktyce polega to na przeskalowaniu i przesunięciu aktywacji w taki sposób, aby ich średnia i wariancja w obrębie mini-batcha pozostawały w ustalonych granicach.

Dla danej partii danych oblicza się średnią i odchylenie standardowe:

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i, \quad \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

a następnie normalizuje się wejście:

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}},$$

gdzie ϵ to mała stała zapobiegająca dzieleniu przez zero. Taka normalizacja sprawia, że aktywacje w każdej warstwie mają średnią równą zero i wariancję równą jeden, co stabilizuje propagację gradientów wstecz i zapobiega zarówno ich zanikaniu, jak i eksplozji.

Jednakże, całkowite znormalizowanie aktywacji mogłoby ograniczyć zdolność sieci do reprezentacji nieliniowych przekształceń. Dlatego w Batch Normalization wprowadza się dodatkowe parametry skalujące i przesuwające, oznaczane odpowiednio jako γ (skala) i β (przesunięcie). Ostateczna postać wyjścia warstwy po normalizacji to:

$$y_i = \gamma \hat{x}_i + \beta$$

Batch Normalization redukuje wpływ zjawiska internal covariate shift – zmiany rozkładu aktywacji w trakcie uczenia, stabilizując tym samym propagację gradientu i umożliwiając stosowanie wyższego współczynnika uczenia. W praktyce BN znacząco przyspiesza trening i często poprawia dokładność końcową modelu.

[10]

Dropout

Drugą powszechnie stosowaną metodą stabilizacji jest Dropout, zaproponowany przez Srivastavę i wsp. w 2014 roku. Technika ta ma na celu ograniczenie zjawiska przeuczenia (ang. overfitting) poprzez losowe wyłączenie części neuronów w trakcie uczenia. W każdej iteracji treningu każdy neuron z prawdopodobieństwem p zostaje „wyłączony” (tj. jego wyjście zostaje ustawione na zero), co można formalnie zapisać jako:

$$\tilde{h}^{(l)} = h^{(l)} \odot r^{(l)}, \quad r^{(l)} \sim \text{Bernoulli}(p)$$

gdzie $\odot$ oznacza mnożenie element po elemencie, a w fazie testowej aktywacje są odpowiednio skalowane:

$$h_{\text{test}}^{(l)} = p h_{\text{train}}^{(l)}$$

aby zachować oczekiwaną wartość wyjść. Dropout można interpretować jako rodzaj losowego „ensemble learning” — w każdej iteracji trenowany jest inny podzbiór sieci, a końcowy model działa jak uśrednienie wielu słabszych klasyfikatorów. W efekcie sieć staje się bardziej odporna na szum w danych i lepiej generalizuje na zbiorze testowym [23].

Learning Rate Scheduling

Proces uczenia sieci neuronowych jest w dużej mierze zależny od wartości współczynnika uczenia η . Stała wartość tego parametru może być nieoptymalna w różnych fazach treningu — zbyt duży krok w pobliżu minimum może powodować oscylacje, a zbyt mały na początku wydłuża proces konwergencji. Z tego powodu stosuje się Learning Rate Scheduling, czyli adaptacyjne strategie zmiany tempa uczenia w czasie.

Najprostszym podejściem jest harmonogram malejący wykładniczo:

$$\eta_t = \eta_0 e^{-kt}$$

gdzie η_0 to początkowy współczynnik uczenia, t — numer epoki, a k — stała określająca szybkość spadku. Popularne są również strategie krokowe (ang. step decay), w których współczynnik uczenia jest zmniejszany skokowo po określonej liczbie epok, oraz metody adaptacyjne, które dostosowują tempo uczenia dynamicznie w zależności od wartości błędu walidacyjnego (ReduceLROnPlateau).

Rozdział 3

Zestawienie użytych rozwiązań

W niniejszym rozdziale przedstawiono zestawienie rozwiązań wykorzystanych w badaniach eksperymentalnych przeprowadzonych w ramach tej pracy. Celem rozdziału jest jednoznaczne zdefiniowanie architektur sieci neuronowych oraz technik uczenia zastosowanych w agentach uczących się gry w Blood Bowl przy użyciu algorytmów uczenia ze wzmocnieniem.

Analizie poddano agentów opartych na algorytmach typu actor-critic, w szczególności A2C oraz PPO, które stanowią wspólną podstawę metod uczenia we wszystkich rozważanych wariantach. W celu zapewnienia porównywalności wyników, algorytmy uczenia, sposób reprezentacji stanu oraz mechanizmy interakcji ze środowiskiem zostały w maksymalnym stopniu ujednolicone. Zmienną badawczą są algorytmy oraz architektury sieci neuronowych.

W kolejnych podrozdziałach opisano ogólną strukturę agentów A2C i PPO, sposób reprezentacji danych wejściowych, a następnie szczegółowo omówiono zastosowane architektury konwolucyjne, w tym warianty oparte na blokach ResNet, Inception oraz ich kombinacjach. Rozdział obejmuje również opis technik stabilizacji i usprawnień procesu uczenia, które okazały się istotne w kontekście złożonego środowiska decyzyjnego gry Blood Bowl.

3.1 Algorytmy aktor-krytyk

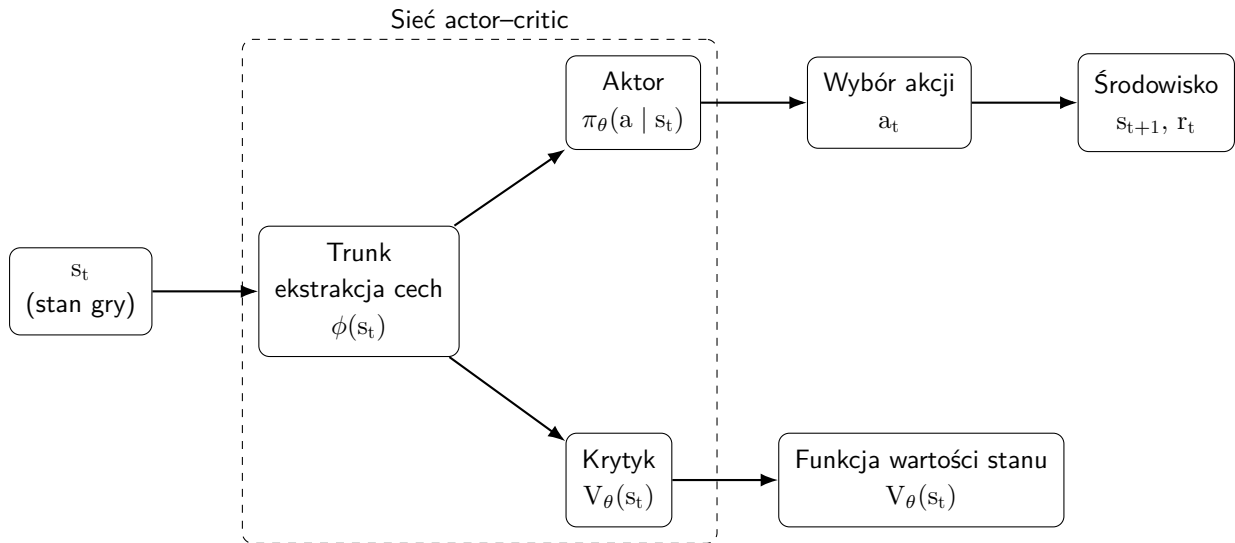
Algorytmy typu aktor-krytyk (ang. *actor-critic*) stanowią klasę metod uczenia ze wzmocnieniem, w których architektura modelu składa się z dwóch współdziałających komponentów: aktora oraz krytyka. Aktor odpowiada za podejmowanie decyzji poprzez aproksymację polityki decyzyjnej, natomiast krytyk pełni rolę oceniającą, aproksymując funkcję wartości stanów.

Tak zdefiniowana struktura umożliwia jednoczesne stosowanie bezpośredniej optymalizacji polityki oraz aproksymacji funkcji wartości. Celem algorytmów aktor-krytyk jest stabilne i efektywne uczenie strategii decyzyjnej w środowiskach o dużej przestrzeni stanów i akcji, takich jak gra Blood Bowl.

Algorytmy aktor-krytyk, w tym A2C oraz PPO, należą do klasy metod *on-policy*. Oznacza to, że dane wykorzystywane do aktualizacji parametrów polityki pochodzą bezpośrednio z interakcji

środowiska z aktualnie uczoną polityką. Trajektorie stan–akcja–nagroda są generowane przez tę samą politykę, która następnie jest optymalizowana w procesie uczenia.

Konsekwencją podejścia on-policy jest brak możliwości wielokrotnego, dowolnego wykorzystania zgromadzonych danych po każdej aktualizacji parametrów polityki, ponieważ zmiana polityki powoduje, że starsze trajektorie przestają być zgodne z jej aktualnym rozkładem. Z jednej strony ogranicza to efektywność wykorzystania próbek, z drugiej jednak zapewnia stabilność procesu uczenia oraz spójność pomiędzy zachowaniem agenta a funkcją celu, co ma kluczowe znaczenie w złożonych, dynamicznych środowiskach decyzyjnych.



Architektura aktor–krytyk (*Actor–Critic*)

Algorytmy A2C oraz PPO należą do klasy metod typu *aktor–krytyk* (actor–critic), które łączą podejście oparte na polityce z estymacją funkcji wartości. W takim podejściu model sieci neuronowej posiada dwie wyspecjalizowane składowe, określane jako *aktor* oraz *krytyk*.

Aktor (ang. *actor*) reprezentuje politykę decyzyjną agenta i odpowiada za wybór akcji na podstawie aktualnego stanu środowiska. Formalnie polityka opisana jest rozkładem prawdopodobieństwa

$$\pi_{\theta}(a | s),$$

gdzie s oznacza stan środowiska, a – akcję, natomiast θ to wektor parametrów sieci. W przypadku przestrzeni akcji dyskretnej, wyjściem aktora jest wektor prawdopodobieństw, z którego próbkujemy się akcją wykonywaną przez agenta.

Krytyk (ang. *critic*) pełni rolę funkcji oceniającej i aproksymuje wartość stanu, czyli oczekiwaną zdyskontowaną sumę przyszłych nagród, jaką agent uzyska, startując ze stanu s i postępując zgodnie z aktualną polityką:

$$V_{\theta}(s) = \mathbb{E}_{\pi_{\theta}} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t = s \right],$$

gdzie r_{t+k} oznacza nagrodę otrzymaną w kroku $t + k$, a $\gamma \in (0, 1)$ jest współczynnikiem dyskontowania. W praktycznych implementacjach krytyk realizowany jest jako druga głowa tej samej sieci neuronowej, współdzieląca część warstw z aktorem.

Rozdzielenie funkcji decyzyjnej i oceniającej znacząco poprawia stabilność procesu uczenia. Krytyk dostarcza aktorowi dodatkowego sygnału uczącego, redukując wariancję estymacji gradientu w porównaniu do metod czysto politycznych, takich jak REINFORCE. Dzięki temu aktualizacje polityki są bardziej stabilne i efektywne, co ma kluczowe znaczenie w środowiskach o dużej przestrzeni stanów i akcji [17].

Funkcja przewagi (*Advantage Function*)

W algorytmach typu aktor–krytyk aktualizacja parametrów polityki opiera się nie na samej sumie nagród, lecz na funkcji przewagi (ang. *advantage function*). Jej zadaniem jest określenie, na ile dana akcja była lepsza lub gorsza od przeciętnego zachowania polityki w danym stanie.

Formalnie funkcja przewagi definiowana jest jako różnica pomiędzy wartością funkcji akcji a wartością stanu:

$$A(s, a) = Q(s, a) - V(s),$$

gdzie $Q(s, a)$ oznacza wartość wykonania akcji a w stanie s , a $V(s)$ jest wartością stanu. Interpretacyjnie $A(s, a) > 0$ oznacza, że akcja a była lepsza od średniej oczekiwanej w stanie s , natomiast $A(s, a) < 0$ wskazuje, że była gorsza.

W praktycznych algorytmach $Q(s, a)$ nie jest obliczane jawnie, ponieważ jego estymacja jest kosztowna i obciążona dużą wariancją. Zamiast tego funkcja przewagi jest aproksymowana na podstawie obserwowanych nagród oraz estymaty krytyka. Typowym przybliżeniem jest n -krokowy zwrot:

$$A_t \approx R_t^{(n)} - V(s_t),$$

gdzie $R_t^{(n)}$ oznacza zdyskontowaną sumę n kolejnych nagród z doszacowaniem wartości stanu końcowego przez krytyka. Bardziej zaawansowanym podejściem jest uogólniona estymacja przewagi (GAE), która wykorzystuje ważoną sumę błędów TD z wielu kroków czasowych.

Zastosowanie funkcji przewagi pełni rolę tzw. *baseline* i prowadzi do istotnego zmniejszenia wariancji estymatora gradientu. W rezultacie polityka uczona jest nie na podstawie surowych nagród, lecz informacji o tym, czy dana decyzja była lepsza czy gorsza od oczekiwań w danym stanie. Mechanizm ten stanowi kluczowy element stabilności i efektywności algorytmów A2C oraz PPO.

Błąd różnicy czasowej (Temporal Difference Error)

Kluczowym pojęciem w algorytmach aktor–krytyk jest błąd różnicy czasowej (ang. *Temporal Difference error*, TD error), który stanowi lokalną miarę niedokładności estymacji funkcji wartości przez krytyka. Błąd TD łączy w sobie idee metod Monte Carlo oraz dynamicznego programowania,

umożliwiając aktualizację wartości stanu na podstawie pojedynczego kroku interakcji ze środowiskiem.

Dla jednego kroku czasowego błąd TD definiuje się jako:

$$\delta_t = r_t + \gamma V_\theta(s_{t+1}) - V_\theta(s_t),$$

gdzie r_t jest nagrodą otrzymaną po wykonaniu akcji a_t w stanie s_t , a γ to współczynnik dyskontowania. Wartość δ_t informuje, czy rzeczywista nagroda oraz przewidywana wartość następnego stanu były większe czy mniejsze od oczekiwań krytyka.

Błąd TD pełni podwójną rolę w algorytmach aktor–krytyk. Po pierwsze, stanowi bezpośredni sygnał uczący dla krytyka, prowadząc do aktualizacji funkcji wartości w kierunku:

$$V_\theta(s_t) \leftarrow V_\theta(s_t) + \alpha \delta_t,$$

gdzie α oznacza współczynnik uczenia. Po drugie, błąd TD jest naturalnym przybliżeniem funkcji przewagi:

$$A_t \approx \delta_t,$$

co pozwala wykorzystać go bezpośrednio w aktualizacji polityki aktora [25].

Uogólnieniem jednokrokowego błędu TD jest estymacja n-krokowa, w której docelowa wartość stanu uwzględnia kilka kolejnych nagród:

$$\delta_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n V_\theta(s_{t+n}) - V_\theta(s_t).$$

Takie podejście redukuje obciążenie estymatora kosztem wzrostu wariancji.

W algorytmach PPO oraz nowoczesnych wariantach A2C często stosuje się uogólnioną estymację przewagi (GAE), która wykorzystuje ważoną sumę błędów TD:

$$A_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l},$$

gdzie parametr $\lambda \in [0, 1]$ pozwala płynnie kontrolować kompromis pomiędzy obciążeniem a wariancją estymatora [21].

Zastosowanie błędu TD umożliwia uczenie online, niską wariancję aktualizacji oraz efektywne wykorzystanie długich trajektorii decyzyjnych, co jest szczególnie istotne w środowiskach o dużej złożoności i rzadkich nagrodach, takich jak Blood Bowl.

3.2 Advanced Actor Critic

A2C, a A3C

Różnica pomiędzy A2C i A3C dotyczy przede wszystkim sposobu równoległego zbierania doświadczeń i aktualizacji parametrów sieci. W A3C (*Asynchronous Advantage Actor–Critic*) wiele niezależnych procesów (aktorów-uczących) równolegle oddziałuje ze środowiskiem i wyznacza gradienty, które są następnie stosowane do wspólnego modelu w sposób asynchroniczny (bez globalnej synchronizacji kroków), co prowadzi do mniej skorelowanych próbek i może stabilizować uczenie [17]. Z kolei A2C (*Advantage Actor–Critic*) stanowi wariant synchroniczny tego podejścia: próbki z wielu równoległych środowisk są agregowane w jedną paczkę (batch), po czym wykonywana jest pojedyncza, wspólna aktualizacja parametrów, co upraszcza implementację oraz czyni przebieg uczenia bardziej deterministycznym przy zachowaniu tej samej ogólnej struktury aktor–krytyk [19].

Generowanie danych uczących: rollout i długość trajektorii w A2C

W algorytmie A2C dane uczące powstają w sposób iteracyjny poprzez równoległe generowanie doświadczeń w wielu instancjach środowiska. W każdej iteracji uruchamia się N środowisk działających równoległe (ang. *vectorized environments*), które wykorzystują tę samą, aktualną politykę π_θ . Następnie z każdego środowiska zbierany jest fragment trajektorii (rollout) o stałej długości T kroków, często oznaczanej jako $t_{\max}$. Otrzymana paczka danych składa się z obserwacji, akcji i nagród:

$$\mathcal{B} = \left\{ \left(s_t^{(i)}, a_t^{(i)}, r_t^{(i)}, s_{t+1}^{(i)} \right) : i = 1, \dots, N, t = 0, \dots, T-1 \right\}.$$

Po zebraniu paczki $\mathcal{B}$ wyznacza się na jej podstawie wielkości wymagane do uczenia, w szczególności przybliżone zwroty (np. n -krokowe) oraz estymaty przewagi A_t , a następnie wykonuje się jedną synchroniczną aktualizację parametrów sieci na całej paczce. Dopiero po tej aktualizacji generowana jest kolejna paczka danych, już z użyciem zaktualizowanej polityki, co jest zgodne z charakterem on-policy A2C [`openai_baselines_a2c`, 17].

Wzory zawierające operator wartości oczekiwanej $\mathbb{E}_t[\cdot]$ interpretowane są w praktyce jako uśrednienie po wszystkich próbkach z paczki rollout:

$$\mathbb{E}_t[f_t] \approx \frac{1}{NT} \sum_{i=1}^N \sum_{t=0}^{T-1} f_t^{(i)}.$$

Takie ujęcie jednoznacznie określa, że estymacja gradientu oraz wartości strat aktora i krytyka opiera się na skończonej próbce doświadczeń zebranych w bieżącej iteracji uczenia.

Funkcja straty w A2C

Uczenie w algorytmie A2C realizowane jest poprzez minimalizację funkcji straty złożonej z komponentów odpowiadających aktorowi (polityce) oraz krytykowi (funkcji wartości), często uzupełnionej o składnik entropii wspierający eksplorację. Taka konstrukcja umożliwia jednocześnie doskonalenie polityki oraz estymację wartości stanu, co stabilizuje proces uczenia w środowiskach o dużej złożoności decyzyjnej [17].

Składnik aktora wynika z twierdzenia o gradiencie polityki i przyjmuje postać ujemnej logarytmicznej wiarygodności wybranej akcji, ważonej estymowaną funkcją przewagi:

$$\mathcal{L}_{\text{actor}}^{\text{A2C}}(\theta) = -\mathbb{E}_t [\log \pi_\theta(a_t | s_t) A_t].$$

Postać ta wzmacnia decyzje przynoszące wyniki ponadprzeciętne ($A_t > 0$) oraz tłumi decyzje gorsze od oczekiwanych ($A_t < 0$) [24].

Składnik krytyka odpowiada za aproksymację funkcji wartości stanu i najczęściej definiowany jest jako błąd średniokwadratowy względem przybliżonej wartości docelowej:

$$\mathcal{L}_{\text{critic}}^{\text{A2C}}(\theta) = \mathbb{E}_t \left[\left(V_\theta(s_t) - \hat{V}_t \right)^2 \right],$$

gdzie $\hat{V}_t$ może oznaczać np. n-krokowy zwrot lub wartość opartą na błędzie TD. Krytyk pełni rolę funkcji bazowej (baseline), zmniejszając wariancję estymatora gradientu aktora i poprawiając stabilność uczenia [17].

W wielu implementacjach A2C stosuje się również bonus entropijny:

$$\mathcal{L}_{\text{entropy}}^{\text{A2C}}(\theta) = -\mathbb{E}_t [\mathcal{H}(\pi_\theta(\cdot | s_t))].$$

Całkowita funkcja straty może być zapisana jako:

$$\mathcal{L}^{\text{A2C}}(\theta) = \mathcal{L}_{\text{actor}}^{\text{A2C}} + c_v \mathcal{L}_{\text{critic}}^{\text{A2C}} - c_e \mathcal{L}_{\text{entropy}}^{\text{A2C}},$$

gdzie c_v oraz c_e są współczynnikami wagowymi regulującymi wpływ krytyka i entropii.

3.3 PPO

Proximal Policy Optimization (PPO) jest algorytmem typu actor-critic zaproponowanym jako praktyczna alternatywa dla metod z ograniczeniem kroku aktualizacji polityki, takich jak TRPO. Główna idea PPO polega na wykonywaniu kilku kroków optymalizacji na tej samej paczce trajektorii, ale w taki sposób, aby nowa polityka nie odsunęła się zbyt gwałtownie od polityki, która wygenerowała dane. Osiąga się to przez zastosowanie funkcji celu opartej na ilorazie prawdopodobieństw (importance sampling ratio) oraz mechanizmie *clippingu*, który ogranicza wpływ zbyt dużych zmian polityki [Schulman2015TRPO, 22].

Funkcja celu PPO (clipped surrogate objective)

Niech θ_{old} oznacza parametry polityki użytej do wygenerowania danych (rolloutu), a θ – parametry aktualizowanej polityki. Definiuje się iloraz:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}.$$

W PPO maksymalizuje się tzw. *surrogate objective* z obcięciem (clipping):

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)],$$

gdzie A_t jest estymatą przewagi (np. z GAE), a ϵ jest hiperparametrem ograniczającym dopuszczalną zmianę polityki w pojedynczej aktualizacji. Mechanizm $\min(\cdot)$ wraz z clip powoduje, że gdy $r_t(\theta)$ wychodzi poza przedział $[1 - \epsilon, 1 + \epsilon]$, dalsze zwiększanie $r_t(\theta)$ nie przynosi już korzyści w funkcji celu, co stabilizuje uczenie i zapobiega destrukcyjnym, zbyt agresywnym krokom optymalizacji [22].

W praktycznych implementacjach PPO optymalizuje się łączną funkcję straty, zawierającą również komponent krytyka (aproxymacja wartości) oraz bonus entropijny:

$$\mathcal{L}^{\text{PPO}}(\theta) = -\mathcal{L}^{\text{CLIP}}(\theta) + c_v \mathbb{E}_t \left[\left(V_{\theta}(s_t) - \hat{V}_t \right)^2 \right] - c_e \mathbb{E}_t [\mathcal{H}(\pi_{\theta}(\cdot | s_t))],$$

gdzie $\hat{V}_t$ oznacza wartość docelową (np. zwrot R_t), a c_v, c_e są wagami części krytyka i entropii. Taka postać odpowiada standardowej praktyce w actor–critic, natomiast różnica PPO polega na zastosowaniu członu $\mathcal{L}^{\text{CLIP}}$ zamiast klasycznej straty polityki bez ograniczenia kroku [22].

Algorytm PPO w ujęciu praktycznym (pętla uczenia)

W odróżnieniu od A2C, w którym po zebraniu rolloutu wykonuje się zwykle pojedynczą aktualizację, PPO wykorzystuje tę samą paczkę danych przez kilka epok optymalizacji (mini-batche). Typowy schemat jest następujący:

1. Zbierz rollout o długości T w N równoległych środowiskach, używając polityki $\pi_{\theta_{\text{old}}}$. Zapisz m.in. (s_t, a_t, r_t) oraz $\log \pi_{\theta_{\text{old}}}(a_t | s_t)$ i estymaty $V_{\theta_{\text{old}}}(s_t)$.
2. Wyznacz $\hat{V}_t$ (zwroty) oraz A_t (np. z GAE) na podstawie zebranej trajektorii [21].
3. Przez K epok wykonuj optymalizację SGD/Adam na losowanych mini-batchach z tej samej paczki, minimalizując $\mathcal{L}^{\text{PPO}}(\theta)$.
4. Ustaw $\theta_{\text{old}} \leftarrow \theta$ i powtórz proces dla kolejnego rolloutu.

Dzięki ilorazowi $r_t(\theta)$ PPO pozostaje metodą bliską on-policy, ale efektywniej wykorzystuje próbki, ponieważ dopuszcza kilka przejść optymalizacyjnych po tych samych danych przy jednoczesnym ograniczeniu odchylenia polityki.

Rodzaj sieci neuronowej w PPO

PPO nie wymaga odrębnego typu sieci względem ogólnej architektury actor–critic: najczęściej stosuje się jedną sieć z *współdzielonym trzonem* (np. CNN przetwarzające wejście przestrzenne), a następnie dwie głowy:

- **głowa polityki (aktor)**: generuje parametry rozkładu $\pi_{\theta}(a | s)$ (w Blood Bowl zwykle logity dla akcji dyskretnych),
- **głowa wartości (krytyk)**: generuje skalar $V_{\theta}(s)$.

Kluczowa różnica implementacyjna względem prostszych wariantów actor–critic polega na tym, że podczas uczenia PPO potrzebuje stabilnego odniesienia do polityki zbierającej dane, czyli przechowywania $\log \pi_{\theta_{\text{old}}}(a_t | s_t)$ (lub równoważnie $\pi_{\theta_{\text{old}}}(a_t | s_t)$), aby obliczać $r_t(\theta)$. Pozostałe elementy (np. maskowanie akcji w przestrzeni dyskretnej) mogą być zastosowane analogicznie jak w innych agentach actor–critic, ponieważ wpływają jedynie na postać $\pi_{\theta}(\cdot | s)$, a nie na samą konstrukcję PPO.

3.4 Współdzielenie części konwolucyjnej

We wszystkich analizowanych architekturach zastosowano wspólną część konwolucyjną (ang. *trunk*), która przetwarza przestrzenne wejście sieci. Dopiero po ekstrakcji cech następuje rozdzielenie na dwie głowy: polityki i wartości. Takie podejście ma kilka istotnych zalet:

- redukuje liczbę parametrów modelu
- pozwala obu głowom korzystać z tej samej reprezentacji stanu,
- znacząco obniża koszty obliczeniowe w porównaniu do tworzenia dwóch oddzielnych sieci.

W praktyce trunk CNN odpowiada za mapowanie danych przestrzennych na wektor cech, który następnie jest łączony z danymi nieprzestrzennymi i przekazywany do końcowych warstw gęstych obu głów.

3.5 Reprezentacja danych wejściowych

W przypadku gry Blood Bowl stan gry ma charakter złożony i heterogeniczny: zawiera zarówno informacje przestrzenne (układ zawodników, lokalizacja piłki, strefy zagrożeń), jak i nieprzestrzenne (licznik tur, aktywny etap procedury gry, dostępne przerzuty). Dlatego konieczne jest zastosowanie reprezentacji umożliwiającej jednocześnie efektywne przetwarzanie danych o różnych charakterystykach strukturalnych.

W pracy przyjęto podział reprezentacji danych wejściowych na dwa niezależne strumienie: strumień przestrzenny, przetwarzany przez konwolucyjne sieci neuronowe, oraz strumień nieprzestrzenny, analizowany za pomocą klasycznych warstw w pełni połączonych.

Strumień Przestrzenny

Strumień przestrzenny stanowi tensor opisujący układ gry na planszy w formie wielowarstwowej mapy cech. Tensor wejściowy ma postać:

$$X \in \mathbb{R}^{C \times H \times W}$$

gdzie

- C oznacz liczbę kanałów cech,
- $H \times W$ odpowiada wymiarom planszy.

Botbowl w formule turniejowej udostępnia 44 kanały. Kanały te zawierają między innymi informacje o

- położenie zawodników drużyny gracza
- położenie zawodników przeciwnika
- parametry zawodników (siła, zwinność, prędkość)
- położenie piłki
- strefa punktowa własnej drużyny
- strefa punktowa przeciwnej drużyny
- pozycje na których można wykonać akcję push
- pozycja wybranego zawodnika
- stany zawodników (ogłuszony, po ruchu)
- szanse na wykonanie bloku

strumień nie przestrzenny

Drugim komponentem reprezentacji danych jest strumień nieprzestrzenny, zawierający wszystkie informacje dotyczące gry, których nie da się w sposób naturalny odwzorować jako mapę dwuwymiarową. Są to np. numer aktualnej tury, flagi opisujące możliwe operacje w danej chwili, dostępne ponowne rzuty obydwu drużyn. Jest on zapisywany w postaci wektora.

$$y \in \mathbb{R}^d$$

gdzie d oznacza liczbę cech.

Strumień nieprzestrzenny jest przetwarzany przez warstwy gęste, które mapują go na reprezentację ukrytą

$$h = \sigma(Wy + b)$$

Integracja strumieni danych

Po przetworzeniu oddzielnych strumieni następuje ich agregacja:

$$z = [\text{flatten}(f_{\text{CNN}}(X)), h]$$

gdzie

- $f_{\text{CNN}}(\cdot)$ oznacza działanie trzonu konwolucji
- h jest przetworzonym wektorem cech skalarnych
- operator $[\cdot]$ oznacza konkatenację wektorową.

Powstały wektor z stanowi skondensowaną reprezentację stanu gry zawierającą zarówno informacje przestrzenne, jak i skalarne. W tej formie trafia on do kolejnych warstw sieci, które generują:

- wartości polityki (logity akcji)
- estymatę funkcji wartości $V_{\theta}(s)$

3.6 Architektura typu CNN z blokiem Inception

W wersji podstawowej sieci konwolucyjne posiadają stałe rozmiary filtrów (np. 3×3), co ogranicza ich zdolność do równoczesnego wychwytywania cech lokalnych i bardziej globalnych. Aby zwiększyć elastyczność i reprezentatywność ekstraktora cech, w niniejszej pracy zastosowano rozszerzenie w postaci bloku Inception, inspirowanego architekturą GoogLeNet

Blok Inception został zaprojektowany tak, aby umożliwić równoległe przetwarzanie tych samych danych za pomocą operatorów konwolucyjnych o różnych rozmiarach filtrów. Każdy z filtrów bada dane w innej skali przestrzennej, co pozwala na lepsze uchwycenie wieloskalowych zależności obecnych na planszy Blood Bowl, takich jak:

- lokalne konfiguracje zawodników (bloki, strefy ataku, pojedyncze zagrożenia)
- średnie struktury pozycyjne (np. zagęszczenia w centrum, formacje ofensywne)
- zależności dalekiego zasięgu (np. ścieżki podań, ustawienia defensywne)

Formalnie, niech wejście do bloku oznaczone będzie jako tensor:

$$X \in \mathbb{R}^{C \times H \times W}$$

Blok inception składa się z zestawu równoległych ścieżek konwolucyjnych, z których każda operuje na filtrach o innym rozmiarze jądra:

$$Y_k = f_k(X), k = 1, 2, 3, \dots, K$$

gdzie $f_k(\cdot)$ to operacja konwolucji o jądrze $k \times k$. Następnie wszystkie przetworzone reprezentacje są łączone wzdłuż wymiaru kanałów:

$$Y = \text{concat}(Y_1, Y_2, \dots, Y_k)$$

Tak powstała cecha końcowa łączy informacje wieloskalowe w jednym tensorze.

Zastosowanie bloków Inception w sieci konwolucyjnej daje kilka istotnych korzyści.

Decyzje podejmowane przez agenta w Blood Bowl zależą zarówno od lokalnych interakcji (np. możliwości bloku), jak i globalnych struktur (np. ustawienie do biegu po przyłożeniu). Architektura Inception pozwala modelować oba rodzaje zależności w ramach jednej warstwy.

Blok Inception jest modularny i dobrze współpracuje z pozostałymi elementami architektur A2C i PPO.

Może być on nakładany sekwencyjnie, zwiększając głębokość modelu. W praktyce zastosowano od kilku do kilkunastu takich bloków w zależności od rozmiaru planszy i dostępnych zasobów obliczeniowych.

Ograniczenia

Mimo licznych zalet, klasyczny Inception stosowany jako wielowarstwowy trunk ma pewne ograniczenia:

- brak mechanizmu kompensacji zanikania gradientu przy głębokich sieciach,
- możliwe zwiększenie kosztów obliczeń przy dużej liczbie kanałów wyjściowych,
- brak informacji rezydualnej, co utrudnia stabilne uczenie w głębokich modelach.

Z tego względu w kolejnych architekturach wprowadzono połączenia rezydualne oraz klasyczną strukturę ResNet, co pozwala poprawić stabilność i jakość ekstrakcji cech.

3.7 Resnet

Kolejną analizowaną architekturą jest sieć konwolucyjna oparta na klasycznym modelu ResNet, charakteryzującym się zastosowaniem bloków rezydualnych o stałej liczbie kanałów. Architektura ta jest uznawana za jedną z najbardziej stabilnych struktur CNN do głębokiej ekstrakcji cech. W przeciwieństwie do bloku Inception, blok rezydualny w ResNet wykorzystuje konwolucje o stałym rozmiarze (zwykle 3×3), jednak równoważy brak wieloskalarności większą głębokością i stabilnością gradientu. Podstawowy blok ResNet składa się z dwóch konwolucji 3×3 z normalizacją i nieliniowością:

$$F(X) = \sigma(\text{BN}(W_2 * \sigma(\text{BN}(W_1 * X))))$$

gdzie,

- W_1, W_2 są filtrami konwolucyjnymi
- BN oznacza normalizację BatchNorm
- σ jest funkcją aktywacji

Wyjście bloku powstaje poprzez dodanie wejścia do przetworzonej reprezentacji:

$$Y = X + F(X)$$

Zalety ResNet

ResNety zostały zaprojektowane specjalnie w celu umożliwienia trenowania bardzo głębokich sieci, nawet powyżej 100 warstw. Połączenia rezydualne stabilizują gradient, co istotnie poprawia możliwości ekstrapolacyjne sieci.

Bloki ResNet dobrze radzą sobie z wykrywaniem średniego i globalnego kontekstu przestrzennego, co jest istotne w ocenie pozycji zawodników, wyznaczaniu ścieżek ruchu i szacowaniu zagrożeń.

Wady ResNet

Choć ResNet jest stabilny i prosty w implementacji, brak mechanizmu wieloskalarności może prowadzić do utraty informacji istotnych dla niektórych sytuacji taktycznych, np. jednoczesnej oceny bloków lokalnych i formacji globalnych. W praktyce jednak jego stabilność i przewidywalność czynią go doskonałą bazą do porównania z bardziej złożonymi architekturami.

3.8 Inception-Residual

Trzecią z analizowanych architektur jest model oparty na blokach Inception, wzbogacony o mechanizm połączeń rezydualnych (residual connections). Rozwiązanie to stanowi połączenie wieloskalowej analizy cech, charakterystycznej dla modułu Inception, z właściwościami stabilizującymi proces uczenia, typowymi dla sieci ResNet. Architektura ta została zaprojektowana w celu zwiększenia głębokości sieci oraz poprawy jakości gradientu podczas optymalizacji, co jest szczególnie istotne w środowiskach o dużej złożoności, takich jak Blood Bowl.

W klasycznym bloku Inception wykorzystuje się równoległe konwolucje o różnych rozmiarach filtrów, które następnie są konkatencjonowane. W obecnej architekturze wynik działania bloku Inception oznaczamy jako:

$$F(X) = \text{concat}(f_1(X), f_3(X), f_5(X))$$

Aby zwiększyć stabilność uczenia i umożliwić efektywne propagowanie gradientu, wprowadzono połączenie rezydualne, które dodaje wejście bloku do jego wyjścia:

$$Y = F(X) + R(X)$$

gdzie $R(X)$ jest transformacją dopasowującą wymiar kanałów wejściowych i wyjściowych. W przypadku zgodnej liczby kanałów stosuje się połączenie tożsamościowe $R(X) = X$. W przypadku rozbieżności stosuje się projekcję:

$$R(X) = W_s * X$$

, gdzie W_s oznacza konwolucję 1×1 dopasowującą wymiar kanałów. Zabieg ten jest analogiczny do rozwiązań stosowanych w ResNet.

Architektura Residual Inception Network łączy komplementarne zalety modułów Inception oraz bloków rezydualnych ResNet, tworząc model o wysokiej zdolności reprezentacyjnej przy jednoczesnej stabilności uczenia. Zastosowanie wielu równoległych filtrów o różnych rozmiarach w bloku Inception umożliwia wydobywanie cech wieloskalowych – od lokalnych interakcji po bardziej globalne wzorce pozycyjne – co jest szczególnie istotne w złożonych środowiskach przestrzennych. Dodanie połączeń rezydualnych znacząco poprawia przepływ gradientu, eliminując problem zanikania sygnału i umożliwiając trenowanie głębszych modeli bez degradacji jakości reprezentacji. Połączenie tych dwóch podejść pozwala sieci zachować elastyczność i ekspresyjność architektury Inception, a jednocześnie wykorzystać właściwości stabilizujące charakterystyczne dla ResNet, prowadząc do skuteczniejszej i bardziej odpornej ekstrakcji cech w złożonych zadaniach decyzyjnych.

3.9 Hyperconnction

Jednym z kluczowych elementów współczesnych architektur głębokiego uczenia są połączenia rezydualne (ang. residual connections), które umożliwiają stabilne trenowanie bardzo głębokich sieci neuronowych. W klasycznej postaci połączenie rezydualne definiowane jest jako suma wyjścia warstwy oraz jej wejścia:

$$h_{l+1} = F(h_l) + h_l \quad (5)$$

gdzie h_l oznacza reprezentację wejściową warstwy l , natomiast $F(\cdot)$ jest transformacją realizowaną przez daną warstwę sieci neuronowej.

Mechanizm ten umożliwia bezpośredni przepływ gradientu pomiędzy warstwami oraz ogranicza problem zanikającego gradientu. Pomimo swojej skuteczności, klasyczne połączenia rezydualne posiadają jednak pewne ograniczenia. W szczególności obserwuje się kompromis pomiędzy zanikaniem gradientu a zjawiskiem kolapsu reprezentacji (ang. representation collapse), w którym kolejne warstwy generują bardzo podobne reprezentacje cech [28].

W celu rozwiązania tego problemu zaproponowano mechanizm *hyper-connections*, który stanowi uogólnienie klasycznych połączeń rezydualnych. Główną ideą tego podejścia jest umożliwienie sieci neuronowej uczenia się siły połączeń pomiędzy reprezentacjami z różnych głębokości sieci zamiast stosowania sztywno zdefiniowanej sumy wejścia i wyjścia warstwy.

Idea hyperconnections

W architekturze hyper-connections zamiast pojedynczego wektora reprezentacji utrzymywany jest zbiór n reprezentacji ukrytych:

$$H_1 = \begin{bmatrix} h_1^{(1)} \\ h_1^{(2)} \\ \vdots \\ h_1^{(n)} \end{bmatrix} \in \mathbb{R}^{n \times d} \quad (6)$$

gdzie:

- n – współczynnik rozszerzenia reprezentacji,
- d – wymiar reprezentacji cech.

Początkowe wejście sieci h_0 jest kopiowane n razy, tworząc macierz:

$$H_0 = \begin{bmatrix} h_0 \\ h_0 \\ \vdots \\ h_0 \end{bmatrix} \quad (7)$$

W każdej warstwie sieci reprezentacje są przekształcane przy użyciu macierzy hyper-connections określającej zależności pomiędzy poszczególnymi reprezentacjami.

Macierz hyper-connections

Mechanizm hyper-connections można opisać przy pomocy macierzy:

$$HC \in \mathbb{R}^{(n+1) \times (n+1)} \quad (8)$$

która określa wagi połączeń pomiędzy reprezentacjami z różnych poziomów sieci.

Macierz ta ma postać:

$$HC = \begin{pmatrix} 0 & \beta_1 & \beta_2 & \dots & \beta_n \\ \alpha_{1,0} & \alpha_{1,1} & \alpha_{1,2} & \dots & \alpha_{1,n} \\ \alpha_{2,0} & \alpha_{2,1} & \alpha_{2,2} & \dots & \alpha_{2,n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \alpha_{n,0} & \alpha_{n,1} & \alpha_{n,2} & \dots & \alpha_{n,n} \end{pmatrix} \quad (9)$$

gdzie:

- β_i określają wpływ wyjścia bieżącej warstwy,
- $\alpha_{i,j}$ określają zależności pomiędzy reprezentacjami.

W przeciwieństwie do klasycznych połączeń rezydualnych wartości te są parametrami uczonymi podczas procesu trenowania sieci.

Obliczanie reprezentacji warstwy

Dla warstwy T reprezentacja wejściowa obliczana jest jako ważona suma reprezentacji z poprzedniego poziomu:

$$h_0 = A_m^T H \quad (10)$$

Następnie transformacja warstwy przyjmuje postać:

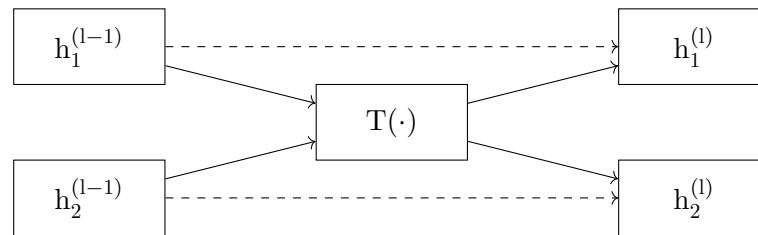
$$\hat{H} = B^T(T(h_0)) + A_r^T H \quad (11)$$

gdzie:

- B określa wpływ wyjścia transformacji,
- A_r odpowiada za połączenia rezydualne.

Diagram działania

Poniższy schemat ilustruje przepływ informacji pomiędzy reprezentacjami w architekturze hyper-connections.



Rysunek 2. Schemat działania mechanizmu hyper-connections.

Mechanizm ten umożliwia jednocześnie integrację informacji pomiędzy różnymi głębokościami sieci oraz wymianę informacji pomiędzy równoległymi reprezentacjami.

Dynamiczne hyper-connections

Rozszerzeniem podstawowego mechanizmu są dynamiczne hyper-connections, w których wagi połączeń zależą od aktualnego wejścia sieci:

$$HC(H) = \begin{pmatrix} 0 & B(H) \\ A_m(H) & A_r(H) \end{pmatrix} \quad (12)$$

Parametry te mogą być obliczane przy pomocy dodatkowej transformacji liniowej:

$$B(H) = s_\beta \tanh(HW_\beta) + B \quad (13)$$

$$A_m(H) = s_\alpha \tanh(HW_m) + A_m \quad (14)$$

$$A_r(H) = s_\alpha \tanh(HW_r) + A_r \quad (15)$$

Takie podejście pozwala sieci dynamicznie dopasowywać siłę połączeń pomiędzy warstwami w zależności od danych wejściowych.

Znaczenie architektoniczne

Istotną własnością hyper-connections jest możliwość reprezentowania różnych struktur architektury sieci neuronowej w ramach jednego modelu. W zależności od wartości macierzy HC sieć może przyjmować charakter:

- sekwencyjny (podobnie jak klasyczna sieć głęboka),
- równoległy (jak w architekturach typu transformer),
- hybrydowy, łączący oba podejścia.

W rezultacie architektura może automatycznie uczyć się optymalnego sposobu organizacji przepływu informacji pomiędzy warstwami sieci neuronowej.

Badania eksperymentalne pokazują, że zastosowanie hyper-connections prowadzi do szybszej konwergencji procesu uczenia oraz poprawy jakości reprezentacji w głębokich modelach [28].

3.10 Rozszerzenia architektury sieci i techniki stabilizacji uczenia

Layer Normalization

Normalizacja warstwowa (ang. Layer Normalization, LN) to technika normalizacji zaproponowana przez Ba i wsp. (2016) jako alternatywa dla normalizacji wsadowej (BN). [1] W przeciwieństwie do BN, która wykorzystuje statystyki z mini-batcha, LN oblicza statystyki normalizacji dla każdej próbki osobno, uwzględniając wszystkie neurony danej warstwy w próbce. Formalnie, dla wektora aktywacji $a^{(l)} = (a_1^{(l)}, a_2^{(l)}, \dots, a_H^{(l)})$ w warstwie l , normalizacja warstwowa oblicza średnią $\mu^{(l)} = \frac{1}{H} \sum_{i=1}^H a_i^{(l)}$ oraz wariancję $\sigma^{2(l)} = \frac{1}{H} \sum_{i=1}^H (a_i^{(l)} - \mu^{(l)})^2$. Następnie każda aktywacja zostaje przeskalowana do postaci znormalizowanej:

$$\hat{a}_i^{(l)} = \frac{a_i^{(l)} - \mu^{(l)}}{\sqrt{\sigma^{2(l)} + \epsilon}}$$

,

gdzie ϵ to niewielka stała zapobiegająca dzieleniu przez zero. Po normalizacji, dla każdego neuronu stosuje się jeszcze parametry adaptacyjne: skalowanie γ_i oraz przesunięcie β_i , analogiczne jak w BN, co daje wyjście $y_i^{(l)} = \gamma_i \hat{a}_i^{(l)} + \beta_i$. Takie podejście powoduje, że wszystkie neurony w warstwie mają jednakowe wartości średniej i wariancji dla danej próbki, ale każda próbka (każdy element batcha) jest normalizowana względem własnych statystyk. W rezultacie znika zależność od rozmiaru batcha – LN

działa poprawnie nawet przy batch size = 1, czyli w trybie online. Co więcej, algorytm normalizacji jest identyczny podczas trenowania i testowania modelu, gdyż statystyki liczone są lokalnie dla próbki (nie ma potrzeby utrzymywania średnich z bieżącej partii czy populacji)

Normalizację warstwową stosuje się zazwyczaj bezpośrednio po warstwie liniowej lub konwolucyjnej, a przed funkcją aktywacji, analogicznie do sposobu użycia Batch Normalization. W testowanych architekturach agentów A2C – takich jak warianty z blokami Inception oraz ich warianty rezydualne – warstwy LayerNorm mogą być stosowana zarówno w trunku konwolucyjnym, jak i w częściach gęstych. W przeciwieństwie do BatchNorm, LN oblicza średnią i wariancję osobno dla każdej próbki, niezależnie od innych elementów batcha. Dzięki temu działa poprawnie nawet przy batch size = 1, co jest szczególnie przydatne w algorytmach on-policy, takich jak PPO i A2C. LN jest także odporny na niestacjonarność rozkładów – nie bazuje na statystykach historycznych, lecz zawsze normalizuje względem bieżącej próbki. Ułatwia to adaptację do zmieniającej się przestrzeni stanów oraz upraszcza implementację, ponieważ nie wymaga przełączania między trybem treningu i ewaluacji. LN lepiej sprawdza się również przy korelowanych danych, typowych dla trajektorii w RL – normalizacja odbywa się niezależnie dla każdej próbki, co zapobiega mieszaniu informacji pomiędzy krokami czasowymi. Badania pokazują, że sieci z LN osiągają stabilniejsze i skuteczniejsze uczenie niż z BatchNorm, zwłaszcza w warunkach zmiennych i silnie skorelowanych danych.

W architekturach głębokich, takich jak Inception-Residual, LN dodatkowo stabilizuje propagację sygnału przez ścieżki rezydualne i przeciwdziała problemom z gradientami. Choć w klasycznej wizji komputerowej preferuje się inne metody, LN okazuje się szczególnie użyteczna w RL, gdzie małe batch size i niestacjonarne dane utrudniają skuteczne użycie BatchNorm.

Parametric Relu

Parametric ReLU (PReLU) to uogólnienie klasycznej funkcji aktywacji ReLU, w którym nachylenie dla ujemnych wartości wejścia nie jest stałe, lecz jest parametryczne (uczony podczas trenowania sieci). Formalnie funkcja PReLU jest zdefiniowana wzorem:

$$f(x) = \begin{cases} x, & x > 0 \\ ax, & x \leq 0 \end{cases} \quad (16)$$

gdzie x oznacza wejście neuronu, zaś a (często oznaczane też α) to współczynnik kontrolujący nachylenie dla ujemnych wartości [7]. W odróżnieniu od zwykłego ReLU (dla którego $a = 0$) wartość a w PReLU jest uczonym parametrem modelu (stąd nazwa Parametric ReLU) i może przyjmować różne wartości w różnych kanałach lub neuronach.

W trakcie przepływu wstecznego (backpropagation) wyznaczane są gradienty zarówno względem wejścia x , jak i względem parametru a . Parametr a jest więc dostosowywany wraz z wagami sieci metodami optymalizacji gradientowej – podczas treningu sieć uczy się optymalnej wartości nachylenia dla ujemnych wartości każdej funkcji aktywacji. W praktyce uczenie parametru a odbywa się standardowo za pomocą algorytmu wstecznej propagacji błędów (np. z zastosowaniem optymalizatora

z momentum). Ważne jest, że nie stosuje się zwykle regularizacji L_2 (tzw. weight decay) na parametr a , aby nie biasować go w kierunku zera (co sprowadziłoby PReLU do zwykłego ReLU). Zamiast tego pozwala się wartości a swobodnie dostosować – badania wskazują, że nawet bez ograniczania zakresu a jego wartości rzadko przekraczają 1 (współczynniki pozostają umiarkowane). Typową inicjalizacją jest nadanie a małej dodatniej wartości (np. 0.25) przed treningiem

He Initialization

Inicjalizacja He (znana też jako Kaiming initialization) to metoda losowego ustawiania początkowych wag sieci neuronowej zaproponowana przez Kaiminga He i wsp. [7] specjalnie z myślą o sieciach z aktywacją ReLU. Wagi każdej warstwy są inicjalizowane jako niezależne zmienne losowe o średniej zero i odpowiednio dobranej wariancji. Formalnie dla wariantu normalnego (rozkład Gaussa) przyjmuje się:

$$w_{ij}^{(l)} \sim N(0, \sigma^2), \text{ gdzie } \sigma^2 = \frac{2}{n_{in}}$$

, czyli wagi pochodzą z rozkładu normalnego o średniej 0 i wariancji $2/n_{in}$ (tu n_{in} oznacza liczbę wejściowych połączeń neuronu, tzw. fan-in danej warstwy). Istnieje też wariant z rozkładu jednostajnego – wówczas wagi losuje się z przedziału symetrycznego:

$$w_{ij}^{(l)} \sim U(-a, +a), \text{ gdzie } a = \sqrt{\frac{6}{n_{in}}}$$

,
tak aby wariancja była równa $2/n_{in}$ (ponieważ dla $U(-a, a)$ wariancja wynosi $a^2/3$, stąd $a = \sqrt{6/n_{in}}$). Metoda He bywa nazywana również MSRA initialization (od Microsoft Research Asia) albo Kaiming initialization, od imienia głównego autora pracy. Celem inicjalizacji He jest zachowanie odpowiedniej skali sygnału (wariancji aktywacji) przy propagacji przez bardzo głęboką sieć. W sieciach głębokich niewłaściwe skalowanie wag może skutkować zanikaniem sygnału albo eksplozją sygnału w kolejnych warstwach.

He i wsp. analizowali wariancję aktywacji warstwy i po warstwie i i wymagali, by iloczyn wariancji sygnału przez kolejne warstwy pozostawał stały (w przybliżeniu równy 1). Warunek ten prowadzi do doboru wariancji wag $\text{Var}(w) \approx 2/n_{in}$, co odpowiada powyższemu wzorowi. Intuicyjnie, współczynnik 2 wynika z faktu, że aktywacja ReLU zeruje średnio połowę wejść – aby utrzymać stałą wariancję sygnału na wyjściu neuronu ReLU, wariancja wag musi być dwa razy większa niż w przypadku funkcji liniowej [7]. Dzięki temu rozwiązaniu amplituda sygnałów nie zanika wraz z głębokością sieci, co ułatwia trenowanie bardzo głębokich modeli.

Inicjalizacja He, a Glorot

Inicjalizacja He została zaprojektowana z myślą o architekturach wykorzystujących niesymetryczne funkcje aktywacji, takie jak ReLU oraz jej rozszerzenia, w szczególności PReLU. W przeciwieństwie

do wcześniejszych metod – jak inicjalizacja Glorota (znana też jako Xavier), która była dostosowana do aktywacji o symetrycznym rozkładzie wyjść (np. funkcji $\tanh$ lub sigmoidalnej) – inicjalizacja He uwzględnia fakt, że funkcje typu ReLU przekazują tylko część dodatnich sygnałów do kolejnych warstw. To asymetryczne zachowanie aktywacji czyni wcześniejsze strategie inicjalizacji mniej skutecznymi w kontekście głębokich modeli wykorzystujących rektyfikację.

W przypadku metody Glorota wagi warstw są dobierane tak, aby równoważyć propagację sygnału do przodu i gradientów wstecz. Wariancja wag zależy od średniej liczby połączeń wejściowych i wyjściowych danego neuronu (fan-in i fan-out), przez co inicjalizacja ta bywa niewystarczająca w sytuacji, gdy aktywacja istotnie zmniejsza amplitudę sygnału. W rezultacie może dojść do szybkiego wygaszenia aktywacji w głębokich warstwach, co utrudnia efektywne trenowanie modelu.

Inicjalizacja He, poprzez skupienie się wyłącznie na liczbie wejść do neuronu, lepiej dopasowuje skalę wag do rektyfikowanych funkcji aktywacji, zachowując stabilność propagowanego sygnału nawet w bardzo głębokich sieciach. Empiryczne wyniki autorów metody potwierdzają, że zastosowanie tej strategii umożliwia skuteczne trenowanie modeli złożonych z wielu warstw, które przy klasycznej inicjalizacji nie zbiegały lub zbiegały bardzo wolno. Ponadto, mimo że metoda He nie została zaprojektowana z myślą o kontroli przepływu gradientu wstecznego, w praktyce pozwala ona również uniknąć problemów eksplodujących lub zanikających gradientów, szczególnie w sieciach wykorzystujących ReLU lub PReLU.

Obsługa maskowania akcji

W algorytmach typu actor-critic kluczowe jest zapewnienie, aby przestrzeń akcji dostępna dla agenta była zgodna z formalnymi ograniczeniami środowiska. W grze Blood Bowl liczba wszystkich możliwych do zakodowania akcji jest bardzo wysoka, lecz w danym stanie jedynie niewielki ich podzbiór stanowi akcje legalne. Pozostałe działania są strukturalnie niedozwolone (np. ruch na zajęte pole, podniesienie nieistniejącej piłki, wykonanie blitzu przez zawodnika, który wykonał już akcję w danej turze).

Aby zapewnić zgodność polityki z regułami środowiska oraz poprawić stabilność procesu uczenia, zastosowano formalny mechanizm *maskowania akcji* (ang. *action masking*). Mechanizm ten polega na modyfikacji wektora logitów polityki poprzez przypisanie logitom odpowiadającym nielegalnym akcjom w stanie s wartości nieskończenie ujemnej.

$$\text{logit}_i = -\infty \quad (17)$$

Po zastosowaniu funkcji softmax

$$\pi_{\theta}(a_i | s) = \frac{\exp(\text{logit}_i)}{\sum_j \exp(\text{logit}_j)}, \quad (18)$$

otrzymujemy w sposób bezpośredni

$$\pi_{\theta}(a_i | s) = 0 \quad \text{jeśli akcja } i \text{ jest nielegalna.} \quad (19)$$

Maskowanie można interpretować jako projekcję rozkładu polityki na zbiór akcji dopuszczalnych:

$$\pi_{\theta}(\cdot | s) \longrightarrow \pi_{\theta}(\cdot | s)|_{\mathcal{A}(s)}, \quad (20)$$

gdzie $\mathcal{A}(s) \subseteq \mathcal{A}$ oznacza zbiór akcji legalnych w stanie s .

Formalnie mechanizm ten można również zapisać jako dodanie wektora maski:

$$\tilde{z}_i = z_i + m_i, \quad (21)$$

gdzie

$$m_i = \begin{cases} 0, & i \in \mathcal{A}(s), \\ -\infty, & i \notin \mathcal{A}(s), \end{cases} \quad (22)$$

a z_i oznacza oryginalne logity polityki.

Tak zdefiniowana transformacja zapewnia, że wartość prawdopodobieństwa każdej akcji spoza zbioru $\mathcal{A}(s)$ równa jest zero, przy jednoczesnym braku wpływu na rozkład prawdopodobieństwa wśród akcji legalnych. W konsekwencji maskowanie poprawia efektywność eksploracji, redukuje wariancję gradientów oraz zapobiega generowaniu trajektorii niezgodnych z regułami gry. Mechanizm ten został zaimplementowany analogicznie do praktyk stosowanych w środowisku BotBowl.

Rozdział 4

Środowisko testowe

Blood Bowl to strategiczna gra planszowa łącząca elementy footballu amerykańskiego oraz gier bitewnych. Rozgrywka toczy się na planszy w formie boiska o wymiarach 26×15 pól. Dwóch graczy prowadzi przeciwne drużyny złożone z figurek (zwanymi zawodnikami) i dąży do zdobycia większej liczby przyłożeń (ang. touchdown) niż przeciwnik. Mecz składa się z dwóch połówek po 8 tur dla każdego gracza. W każdej turze trener może aktywować wielu swoich zawodników, wykonując nimi szereg akcji na boisku. Do podstawowych typów akcji należą m.in.: ruch (przesunięcie zawodnika o kilka pól zależnie od jego parametru szybkości), podanie piłki, przekazanie piłki innemu zawodnikowi (handoff), blok (atak na sąsiadującego przeciwnika w próbie powalenia go), blitz (połączenie ruchu i ataku), a nawet sfaulowanie leżącego przeciwnika (foul). Gdy zawodnik z piłką dotrze do strefy punktowej rywala, jego drużyna zdobywa punkt. Po upływie ustalonej liczby tur mecz kończy się, a wygrywa drużyna z większą liczbą przyłożeń.

Istotnym elementem Blood Bowl jest losowość oraz wynikające z niej zarządzanie ryzykiem. Większość istotnych akcji – takich jak bloki, uniki, podania czy chwyt piłki – wymaga powodzenia testu kośćmi. Szanse zależą od atrybutów zawodników (np. Siły przy blokowaniu, Zręczności przy chwytaniu czy uniku) oraz posiadanych umiejętności specjalnych (np. Dodge, Block). Niepowodzenie testu zwykle oznacza natychmiastową utratę tury (tzw. turnover). W związku z tym trenerzy muszą rozważnie planować kolejność działań – zazwyczaj zaczynają od ruchów bezpiecznych (niewymagających rzutów lub z bardzo dużą szansą powodzenia), odkładając ryzykowne akcje na koniec tury. Gra przyznaje też ograniczoną liczbę żetonów przerzutów, które pozwalają powtórzyć nieudany rzut kością, co dodaje element decyzyjny dotyczący zarządzania zasobami szczęścia. Stan gry w Blood Bowl jest złożony i bogaty: obejmuje on nie tylko położenie wszystkich zawodników na planszy, ale również ich stan (stojący, powalony, ogłuszony itp.), posiadane atrybuty i umiejętności, stan piłki, warunki pogodowe, licznik tur, rezerwy i kontuzje graczy w drużynach, dostępne przerzuty i inne elementy. W odróżnieniu od gier takich jak Go czy szachy, reprezentacja stanu w Blood Bowl ma charakter wielowymiarowy przestrzennie oraz nieprzestrzennie, a przestrzeń akcji nie jest stała – dostępne ruchy zależą od kontekstu sytuacyjnego każdego kroku.



4.1 Bloodbowl jako benchmark dla uczenia ze wzmocnieniem.

Blood Bowl jest uważany za kolejne wielkie wyzwanie dla sztucznej inteligencji w grach planszowych njustesen.github.io. Kilka cech tej gry sprawia, że stanowi ona wartościowe środowisko testowe dla metod uczenia ze wzmocnieniem i sieci neuronowych:

1. Wysoki branching factor – Każda tura oferuje olbrzymią liczbę możliwych sekwencji ruchów. Trener może poruszać wieloma zawodnikami w dowolnej kolejności, a każdy z nich może wykonać kilka akcji. Średni rozgałęźnik (branching factor) pojedynczej akcji zawodnika szacuje się na ok. 10^5 możliwych przebiegów. W rezultacie pojedyncza tura (maksymalnie 10 zawodników) może rozwidlać się nawet na rząd 10^{50} możliwych ciągów akcji. Dla porównania, w szachach średni branching factor wynosi ok. 30, a w go ok. 300. Tak ogromna przestrzeń decyzji przekracza możliwości tradycyjnych algorytmów przeszukiwania i stanowi poważne wyzwanie dla metod RL.
2. Nieregularne drzewa decyzyjne – W Blood Bowl długość tury oraz dostępne akcje nie są z góry ustalone. Z jednej strony, lista dozwolonych ruchów w danym momencie zależy od stanu gry (np. tylko niektórzy zawodnicy mogą wykonać określone akcje, możliwe pola ruchu są

ograniczone zasięgiem, itp.), co oznacza dynamicznie zmieniającą się przestrzeń akcji. Z drugiej strony, wskutek losowości tura gracza może zakończyć się wcześniej (np. nieudany rzut kończy turę), przez co drzewo rozgrywki ma zmienną głębokość gałęzi. Strukturę decyzyjną Blood Bowl można określić jako niestandardową lub zagnieżdżoną: tura składa się z serii ruchów zawodników (podtur), z których każdy z kolei bywa sekwencją kilku akcji (np. ruch przez kilka pól, a następnie blok). Taka nieregularność utrudnia zastosowanie klasycznych algorytmów planowania, które zakładają stałą liczbę ruchów na turę i jednolitą strukturę drzewa.

3. Wieloetapowy charakter rozgrywki – Blood Bowl wymaga długoterminowego planowania oraz podejmowania szeregu skorelowanych decyzji w ramach jednej tury i całego meczu. Pełna rozgrywka to kilkanaście tur na stronę, łącznie około 1600 kroków decyzyjnych według szacunków. Punkty (przyłożenia) są zdobywane rzadko – w wielu partiach pada bardzo mało goli, zwłaszcza przy losowej lub słabej grze zdarza się remis 0:0. Taka długa horyzontalnie sekwencja decyzji ze sporadyczną nagrodą (sparse reward) powoduje, że algorytmy oparte na przeszukiwaniu oraz klasyczne metody RL mają trudność z uzyskaniem sygnału uczącego. Konieczne jest więc planowanie wieloetapowe – zarówno taktyczne (w obrębie jednej tury, optymalna kolejność akcji pod ryzykiem turnover), jak i strategiczne (przez cały mecz, np. zabezpieczenie piłki, kontrola czasu gry itp.).
4. Aspekty przestrzenne gry – Gra ma wyraźny charakter przestrzenny: położenie zawodników na planszy i kontrola terytorium (strefy ataku, krycie przeciwników, droga do pola punktowego) są kluczowe dla sukcesu. Stan gry można przedstawić w formie siatki cech (mapy zajętości pól, stref wpływu, pozycji piłki, itp.), co przypomina reprezentacje stanów znane z gier takich jak StarCraft II. Równocześnie jednak Blood Bowl posiada elementy symboliczne i atrybutowe (cechy postaci, liczniki, efekty), których nie da się ująć wyłącznie na obrazowej planszy. Ta kombinacja informacji przestrzennej i nieprzestrzennej wymaga od agentów uczenia się zarówno rozumienia układów geometrycznych na polu gry, jak i śledzenia zmiennych nienumerycznych (punkty, przerzuty, statusy). Stanowi to dobre pole do badań nad połączeniem sieci konwolucyjnych (do map cech) z sieciami gęstymi lub mechanizmami uwagi, analogicznie do podejść stosowanych w środowisku StarCraft II Learning Environment (SC2LE) [27].

4.2 Środowisko BotBowl

Aby badać opisane wyżej zagadnienia, stworzono BotBowl – otwartoźródłowe środowisko symulujące pełne zasady Blood Bowl, znane też jako Fantasy Football AI (FFAI). Platforma ta dostarcza szereg funkcjonalności wspierających efektywne szkolenie agentów sztucznej inteligencji.

BotBowl udostępnia interfejs zgodny z popularnym schematem OpenAI Gym, co ułatwia integrację z istniejącymi algorytmami RL. Środowisko zostało zaimplementowane w języku Python, dzięki czemu można bezpośrednio korzystać z bibliotek uczenia maszynowego, a sam interfejs obejmuje standardowe metody jak `reset()` (inicjalizacja nowej gry) oraz `step(akcja)` (wykonanie kroku).

Framework BotBowl umożliwia łatwe tworzenie i włączenie własnych agentów. Udostępniane jest API wysokiego poziomu: wystarczy zaimplementować klasę bota dziedziczącą po abstrakcyjnej klasie Agent (z metodami m.in. `act()` definiującą decyzje na podstawie stanu gry). Następnie takiego bota można zarejestrować w systemie pod unikalną nazwą i używać go w symulacjach lub turniejach. BotBowl zawiera też przykładowe zaimplementowane boty (np. losowy RandomBot, heurystyczny GrodBot) mogące służyć jako punkt wyjścia lub przeciwnicy do testów.

Środowisko pozwala na resetowanie stanu gry oraz rozgrywanie meczów między wybranymi agentami w trybie symulowanym bez interakcji użytkownika. Możliwe jest również równoległe uruchamianie wielu instancji gry (multi-env) w celu przyspieszenia treningu. Dodatkowo BotBowl zawiera forward model (silnik przewidywania następstw akcji), co umożliwia zastosowanie metod planowania czy przeszukiwania drzewa gry (np. MCTS) obok czystego RL. Wbudowane funkcje turniejowe pozwalają automatycznie rozegrać serię gier między botami, kontrolując limity czasu na turę oraz wykrywając ewentualne zawieszenia lub awarie agentów (co jest istotne przy treningu i testowaniu na szeroką skalę).

Stan rozgrywki zwracany przez środowisko jest podzielony na część przestrzenną i nieprzestrzenną. Część przestrzenna to tensor wymiaru (L, H, W) , gdzie $H \times W$ odpowiada polu gry (wraz z ewentualnymi obrzeżami), zaś L to liczba warstw cech. Domyślnie dostarczanych jest kilkadziesiąt warstw, m.in. zajętość pola przez zawodników, strefy ataku, pozycja piłki, pola dostępne do ruchu, prawdopodobieństwa powodzenia rzutów itp. . Część nieprzestrzenna to wektor liczbowy zawierający inne informacje o stanie gry – np. liczba tur, punkty, pozostałe przerzuty obu drużyn, aktualna faza/procedura gry – a także wskazania, które typy akcji są obecnie dostępne. Co ważne, BotBowl dostarcza również maskę dozwolonych akcji, dzięki czemu agent otrzymuje informację, które konkretne akcje (z ogromnej puli wszystkich możliwych) są w danym momencie legalne. Przestrzeń wszystkich potencjalnych akcji jest bowiem bardzo duża – rzędu tysięcy możliwych kodowanych decyzji – jednak w danej sytuacji tylko niewielki podzbiór ruchów wchodzi w grę. Maskę (oraz alternatywnie struktura danych `game.state.available_actions`) pozwala ograniczyć wybór do ruchów legalnych, co znacznie ułatwia trenowanie modelu i konwergencję algorytmów RL.

Podsumowując, Blood Bowl dostarcza unikalne, wymagające środowisko o wysokiej złożoności, które dzięki platformie BotBowl jest dostępne do eksperymentów z algorytmami uczenia ze wzmocnieniem. Połączenie czynników takich jak ogromny rozgałęźnik decyzyjny, losowość i wieloetapowość decyzji oraz potrzeba percepcji przestrzennej czyni z niego atrakcyjny poligon doświadczalny dla nowoczesnych metod AI. BotBowl zapewnia zaś odpowiednie narzędzia – od interfejsu Gym, poprzez bogatą reprezentację stanów i mechanizmy ułatwiające implementację własnych agentów, aż po możliwość symulacji gier na masową skalę – aby skutecznie trenować i testować modele uczące się gry w Blood Bowl.(oraz dokumentacja BotBowl).

Rozdział 5

Wyniki

W tym rozdziale przedstawiono wyniki procesu uczenia w środowisku Botbowl porównywanych sieci neuronowych. Celem przeprowadzonych eksperymentów było określenie wpływu architektury sieci neuronowej na zdolność agenta do nauki skutecznej polityki gry w środowisku o bardzo dużej przestrzeni stanów oraz wysokim współczynniku branching factor.

Środowisko Blood Bowl jest szczególnie wymagające z punktu widzenia uczenia ze wzmocnieniem. W wielu stanach gry agent ma do wyboru bardzo dużą liczbę możliwych akcji, a konsekwencje podjętych decyzji mogą ujawniać się dopiero po wielu kolejnych krokach. Powoduje to, że proces uczenia charakteryzuje się dużą wariancją, a stabilność algorytmu oraz zdolność modelu do ekstrakcji istotnych cech ze stanu gry odgrywają kluczową rolę.

Zastosowane miary jakości

W trakcie uczenia zbierane były 3 statystyki określające postęp uczenia.

Wykresy *Win rate*, tj. odsetek zwycięstw w rozgrywkach ewaluacyjnych. Wartość około 0.5 oznacza poziom zbliżony do przeciwnika (w przybliżeniu "remis" sił), natomiast wartości istotnie wyższe od 0.5 wskazują na realną poprawę skuteczności agenta.

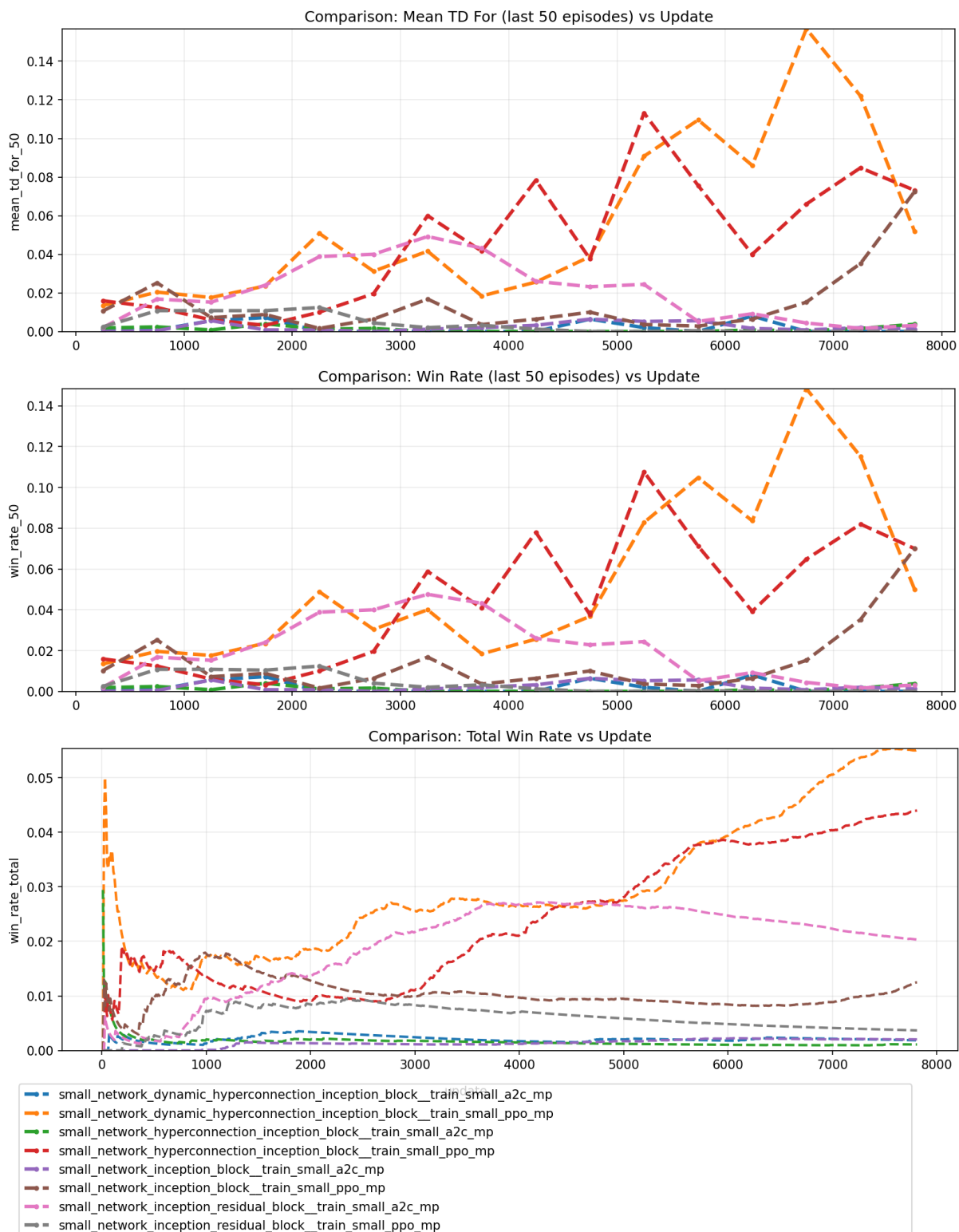
Wykres *Reward* przedstawia średnią (lub uśrednianą w oknie) nagrodę uzyskiwaną przez agenta w trakcie treningu. Ponieważ w Blood Bowl nagrody są w dużej mierze rzadkie i silnie zależne od długoterminowych konsekwencji decyzji, sama wartość *Reward* może charakteryzować się dużą wariancją i nie zawsze wprost przekłada się na poprawę jakości gry.

Wykres *TD/Episode* raportuje liczbę przyłożeń na epizod osobno dla uczącego się agenta (*Learner*) oraz przeciwnika (*Opponent*). Miara ta jest bardziej bezpośrednio związana z przebiegiem meczu niż *Reward*, jednak również bywa rzadka (w wielu partiach liczba przyłożeń jest niska), przez co pojawiają się skoki zamiast gładkich trendów.

Pośród przedstawionych miar najważniejszym wskaźnikiem jakości wyuczonej polityki jest *Win rate*, ponieważ bezpośrednio odzwierciedla skuteczność agenta w rozgrywce przeciwko przeciwnikowi testowemu.

Wyniki

Na poniższych wykresach przedstawiono porównanie wyników dla wszystkich analizowanych konfiguracji architektur sieci neuronowych. Wykresy prezentują zależność analizowanych metryk od liczby iteracji treningowych.

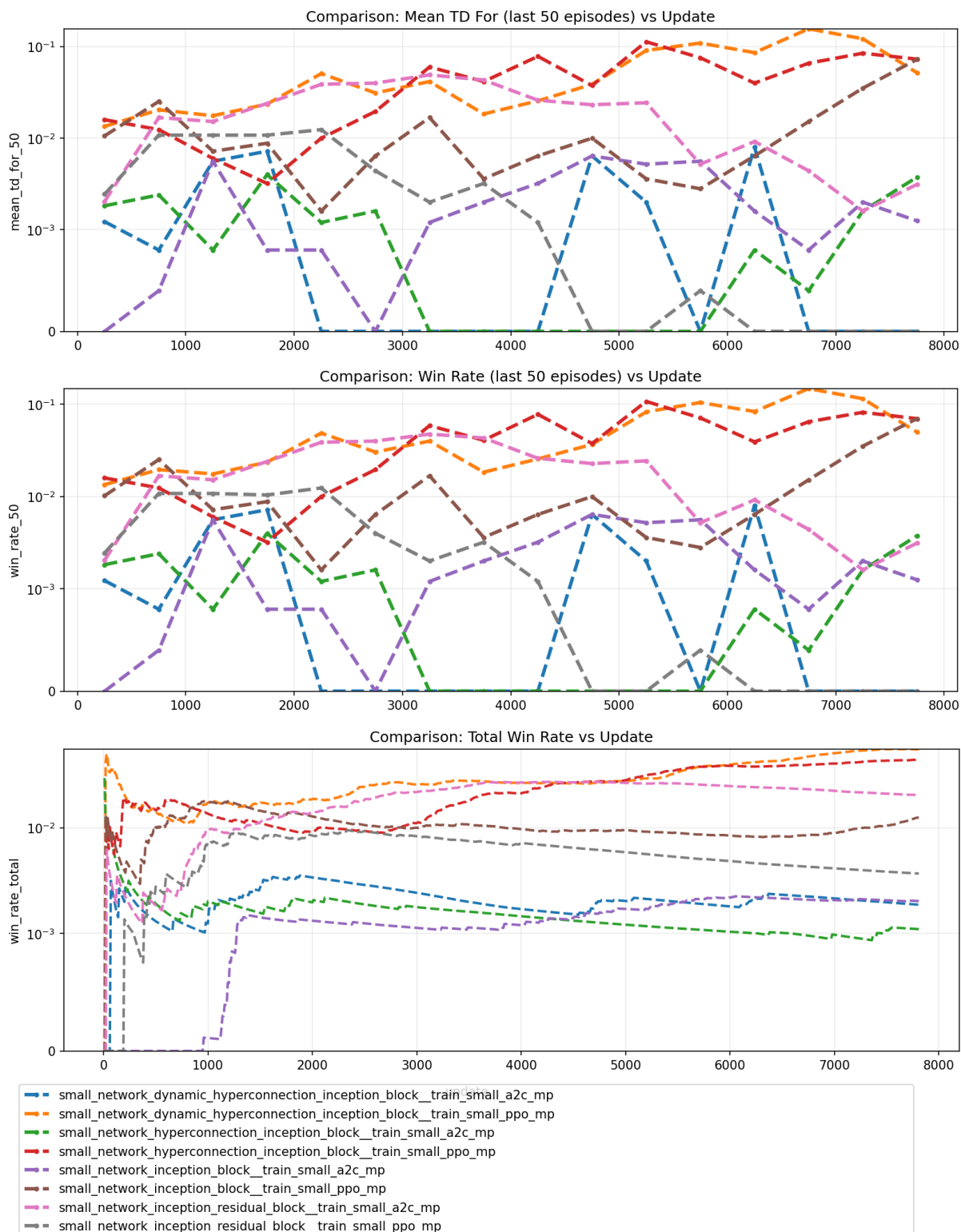


Pierwszy z powyższych wykresów przedstawia średnią liczbę przyłożeń na epizod dla ostatnich 50 rozegranych epizodów. Wraz z postępem treningu można zaobserwować wzrost tej wartości dla części

analizowanych konfiguracji, co wskazuje na stopniową poprawę zdolności agenta do skutecznego prowadzenia akcji ofensywnych.

Drugi wykres przedstawia średnią wartość wskaźnika *Win rate* obliczaną w oknie obejmującym ostatnie 50 epizodów. Metryka ta pozwala obserwować krótkoterminowe zmiany jakości gry agenta w trakcie treningu. W przypadku części architektur widoczny jest systematyczny wzrost wartości tej miary, co wskazuje na stopniową poprawę wyuczonej polityki.

Trzeci wykres przedstawia całkowity wskaźnik zwycięstw w funkcji liczby iteracji treningowych. W przeciwieństwie do poprzedniego wykresu, miara ta ma charakter bardziej globalny i pokazuje długoterminowy trend skuteczności agenta. Wraz z postępem uczenia można zaobserwować stopniowy wzrost tej wartości dla najlepszych konfiguracji architektur.



Dodatkowo przedstawiono te same wykresy w skali logarytmicznej. Pozwala to lepiej uwidocznić różnice pomiędzy poszczególnymi konfiguracjami w początkowych etapach treningu, gdy wartości analizowanych metryk są jeszcze stosunkowo niewielkie.

Zastosowanie skali logarytmicznej umożliwia również łatwiejszą obserwację dynamiki wzrostu analizowanych wskaźników w kolejnych etapach procesu uczenia.

Legenda przedstawiona powyżej umożliwia identyfikację poszczególnych konfiguracji architektur analizowanych w eksperymentach.

Ograniczenia eksperymentu

Należy zauważyć, że przeprowadzone eksperymenty posiadają pewne ograniczenia. Przede wszystkim liczba iteracji treningowych była ograniczona przez dostępne zasoby obliczeniowe, co mogło wpłynąć na możliwość pełnej konwergencji niektórych modeli.

Ponadto środowisko Blood Bowl charakteryzuje się znaczną losowością wynikającą z mechaniki gry, w szczególności z rzutów kośćmi. Powoduje to dużą wariancję uzyskiwanych wyników, co utrudnia jednoznaczną ocenę jakości wyuczonej polityki na podstawie pojedynczych przebiegów treningu.

Dodatkowo wszystkie eksperymenty zostały przeprowadzone przeciwko określonemu przeciwnikowi testowemu. W praktyce skuteczność wyuczonej strategii może zależeć od stylu gry przeciwnika, dlatego dalsze badania mogłyby obejmować ewaluację agentów przeciwko większej liczbie różnych strategii.

Wnioski

Na podstawie przeprowadzonych eksperymentów można sformułować następujące wnioski:

- Architektura sieci neuronowej ma istotny wpływ na zdolność agenta do nauki skutecznej polityki gry w środowisku Blood Bowl.
- W środowiskach o dużej przestrzeni stanów oraz wysokim branching factor szczególnie ważna jest zdolność modelu do efektywnej ekstrakcji cech opisujących stan gry.
- Analiza przebiegu metryk *Win rate* oraz *TD/Episode* wskazuje, że część architektur umożliwia stopniową poprawę jakości strategii agenta wraz z postępem treningu.
- Uzyskane wyniki wskazują również potencjalne kierunki dalszych badań, takie jak zastosowanie głębszych architektur konwolucyjnych, mechanizmów uwagi (attention) lub bardziej zaawansowanych algorytmów uczenia ze wzmocnieniem.

Bibliografia

- [1] Ba, J. L., Kiros, J. R. i Hinton, G. E., *Layer Normalization*, 2016. arXiv: 1607.06450 [stat.ML]. adr.: <https://arxiv.org/abs/1607.06450>.
- [2] Bishop, C. M., *Pattern Recognition and Machine Learning*. New York: Springer, 2006, ISBN: 978-0-387-31073-2.
- [3] Cybenko, G., „Approximation by superpositions of a sigmoidal function”, *Mathematics of Control, Signals and Systems*, t. 2, nr. 4, s. 303–314, grud. 1989, ISSN: 1435-568X. DOI: 10.1007/BF02551274. adr.: <https://doi.org/10.1007/BF02551274>.
- [4] Dai, Z. i Heckel, R., *Channel Normalization in Convolutional Neural Network avoids Vanishing Gradients*, lip. 2019. DOI: 10.48550/arXiv.1907.09539.
- [5] Goodfellow, I., Bengio, Y. i Courville, A., *Deep Learning*. MIT Press, 2016. adr.: <http://www.deeplearningbook.org>.
- [6] Goodfellow, I. J. i in., *Generative Adversarial Networks*, 2014. arXiv: 1406.2661 [stat.ML]. adr.: <https://arxiv.org/abs/1406.2661>.
- [7] He, K., Zhang, X., Ren, S. i Sun, J., *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification*, 2015. arXiv: 1502.01852 [cs.CV]. adr.: <https://arxiv.org/abs/1502.01852>.
- [8] Hinton, G., Srivastava, N. i Swersky, K., „Neural networks for machine learning lecture 6a overview of mini-batch gradient descent”, *Cited on*, t. 14, nr. 8, s. 2, 2012.
- [9] Hornik, K., Stinchcombe, M. i White, H., „Multilayer feedforward networks are universal approximators”, *Neural Networks*, t. 2, nr. 5, s. 359–366, 1989, ISSN: 0893-6080. DOI: [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8). adr.: <https://www.sciencedirect.com/science/article/pii/0893608089900208>.
- [10] Ioffe, S. i Szegedy, C., „Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift”, w *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, 2015, s. 448–456.
- [11] Kingma, D. P. i Ba, J., „Adam: A Method for Stochastic Optimization”, *International Conference on Learning Representations (ICLR)*, 2015. arXiv: 1412.6980 [cs.LG]. adr.: <https://arxiv.org/abs/1412.6980>.
- [12] Kohonen, T., „The self-organizing map”, *Proceedings of the IEEE*, t. 78, nr. 9, s. 1464–1480, 1990. DOI: 10.1109/5.58325.

- [13] LeCun, Y., Bottou, L., Bengio, Y. i Haffner, P., „Gradient-based learning applied to document recognition”, *Proceedings of the IEEE*, t. 86, nr. 11, s. 2278–2324, 1998.
- [14] McCulloch, W. S. i Pitts, W., „A logical calculus of the ideas immanent in nervous activity”, *The bulletin of mathematical biophysics*, t. 5, nr. 4, s. 115–133, grud. 1943, ISSN: 1522-9602. DOI: 10.1007/BF02478259. adr.: <https://doi.org/10.1007/BF02478259>.
- [15] Mhaskar, H. N. i Poggio, T., „Function approximation by deep networks”, *Communications on Pure and Applied Analysis*, t. 19, nr. 8, s. 4085–4095, 2020, ISSN: 1534-0392. DOI: 10.3934/cpaa.2020181. adr.: <https://www.aims sciences.org/article/id/519d7cfc-913a-45ea-9a9b-cee4dd60e31b>.
- [16] Minsky, M. L. i Papert, S. A., *Perceptrons: expanded edition*. Cambridge, MA, USA: MIT Press, 1988, ISBN: 0262631113.
- [17] Mnih, V. e. a., „Asynchronous Methods for Deep Reinforcement Learning”, *Proceedings of the 33rd International Conference on Machine Learning*, 2016.
- [18] Nesterov, Y., „A method of solving a convex programming problem with convergence rate $O(1/k^2)$ ”, *Soviet Mathematics Doklady*, t. 27, s. 372–376, 1983.
- [19] OpenAI, *OpenAI Baselines: ACKTR & A2C*, OpenAI, 2017. adr.: <https://openai.com/index/openai-baselines-acktr-a2c/>.
- [20] Rolnick, D. i Tegmark, M., „The power of deeper networks for expressing natural functions”, maj 2017. DOI: 10.48550/arXiv.1705.05502.
- [21] Schulman, J. e. a., „High-Dimensional Continuous Control Using Generalized Advantage Estimation”, *International Conference on Learning Representations*, 2016.
- [22] Schulman, J. e. a., „Proximal Policy Optimization Algorithms”, *arXiv preprint arXiv:1707.06347*, 2017.
- [23] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. i Salakhutdinov, R., „Dropout: A Simple Way to Prevent Neural Networks from Overfitting”, *Journal of Machine Learning Research*, t. 15, nr. 1, s. 1929–1958, 2014.
- [24] Sutton, R. S. i Barto, A. G., *Temporal-difference learning*. MIT Press Cambridge, MA, USA, 2018, s. 131–132.
- [25] Sutton, R. S. i Barto, A. G., *Reinforcement Learning: An Introduction*, 2nd. MIT Press, 2018.
- [26] Telgarsky, M., „Benefits of depth in neural networks”, w *Conference on learning theory*, PMLR, 2016, s. 1517–1539.
- [27] Vinyals, O., Babuschkin, I., Czarnecki, W. M. i in., „StarCraft II: A New Challenge for Reinforcement Learning”, *arXiv preprint arXiv:1708.04782*, 2017.
- [28] Zhu, T. i in., „Hyper-Connections”, *arXiv preprint arXiv:2409.19606*, 2024.

Wykaz skrótów i symboli

Spis rysunków

1	Porównanie płytkiej i głębokiej sieci neuronowej z wyrównanym położeniem neuronów wyjściowych.	15
2	Schemat działania mechanizmu hyper-connections.	43

Spis tabel

1	Porównanie funkcji aktywacji oraz ich pochodnych	14
---	--	----